Introducción Deep Learning



Aplicaciones del aprendizaje profundo en el análisis de grandes volúmenes de datos.
Especialización en Ciencia de Datos.

Mg. Diego Encinas - Ing. Román Bond

Agenda

- ¿Qué es Deep Learning?
- Redes Neuronales Artificiales - ANN
- ANN - Perceptrón
- ANN - Funciones de Activación
- ANN - Funciones de Activación
- Python y Deep Learning
- Ejemplos!
- ANN
- CNN - Redes Neuronales Convolucionales
- Ejemplos!

¿Qué es Deep Learning?

Es una de las técnicas utilizadas en Machine Learning.

Con Deep Learning se busca extraer características de datos sin procesar, utilizando diferentes capas. El objetivo es identificar los aspectos más representativos y relevantes de los datos.

¿Qué es Deep Learning?

Deep Learning nace como un subconjunto de técnicas de Machine Learning. Mediante el uso de redes neuronales (redes neuronales profundas) se puede simular el proceso de toma de decisión del cerebro humano.

¿Qué es Deep Learning?

¿Y la diferencia con Machine Learning?

Depende de la complejidad y estructura de las redes neuronales utilizadas. En Machine Learning se suele trabajar con redes neuronales simples, de una o dos capas. En Deep Learning, la arquitectura considera múltiples capas (cientas si es necesario). Por ejemplo ResNet.

fuente: https://www.ibm.com/think/topics/deep-learning

ResNet: https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

¿Qué es Deep Learning?

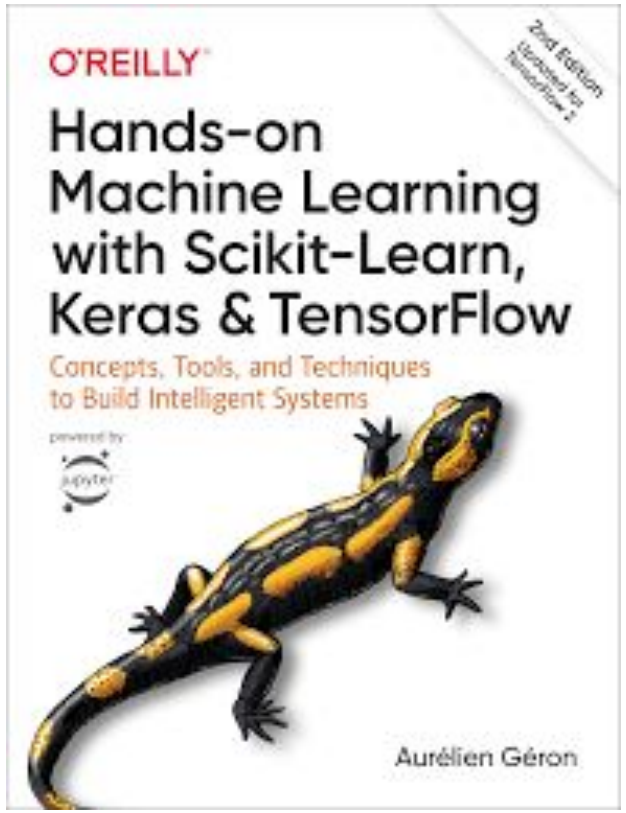
¿Por dónde empezamos?

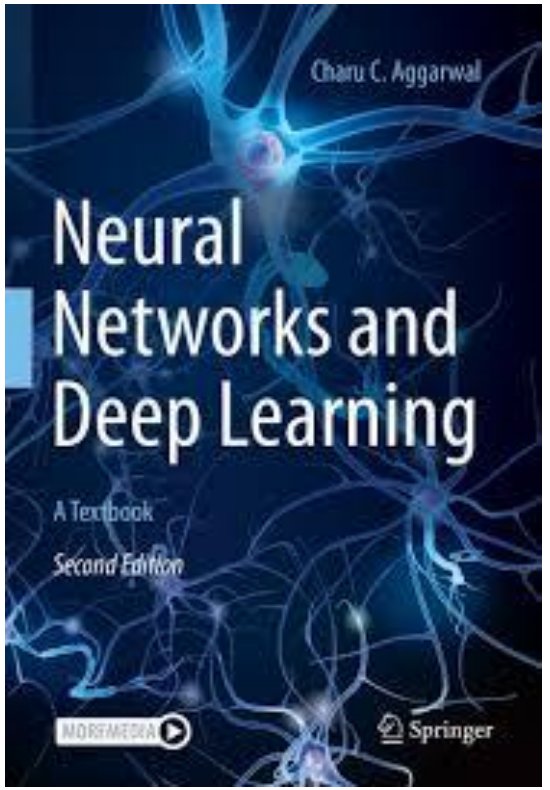




¿Qué es Deep Learning?

¿Por dónde empezamos? Mejor dicho, ¿con qué continuamos?





Redes Neuronales Artificiales - ANN

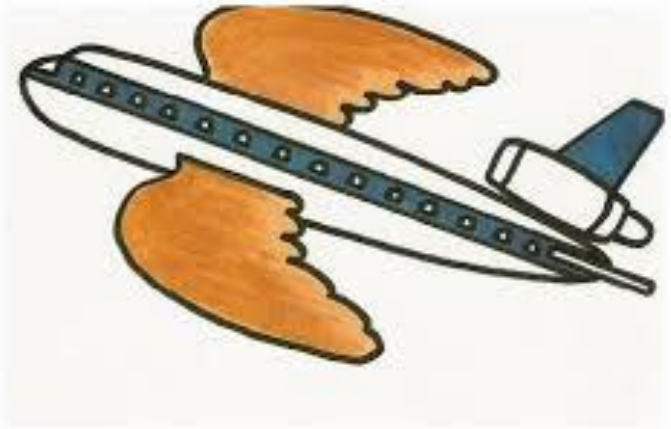
La naturaleza ha sido fuente de inspiración de numerosos inventos y desarrollos. Por ejemplo, las aves nos inspiraron a volar y dominar el vuelo.

Esta lógica es la que sirve como puntapié para la creación de Redes Neuronales Artificiales: un modelo de Machine Learning inspirado en el funcionamiento de las redes neuronales biológicas del cerebro.

Redes Neuronales Artificiales - ANN

Si bien los aviones toman inspiración de las aves, no es necesario que muevan sus alas. De forma muy similar, las ANN son diferentes de las redes neuronales biológicas.

La palabra clave aquí es *inspiración.*

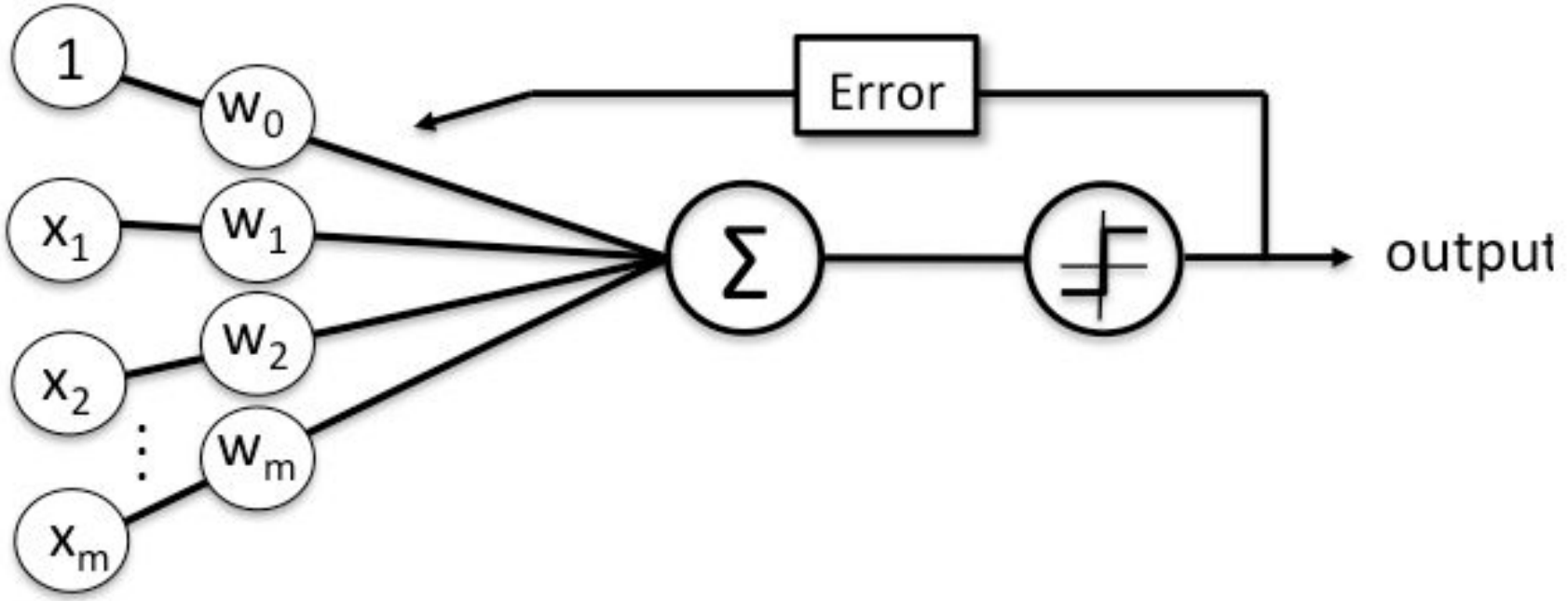


Redes Neuronales Artificiales - ANN

Las ANN son el corazón de Deep Learning. Permiten lidiar con problemas complejos y de gran escala que presenta Machine Learning al momento de realizar clasificaciones, regresiones, reconocimiento de voz, etc.

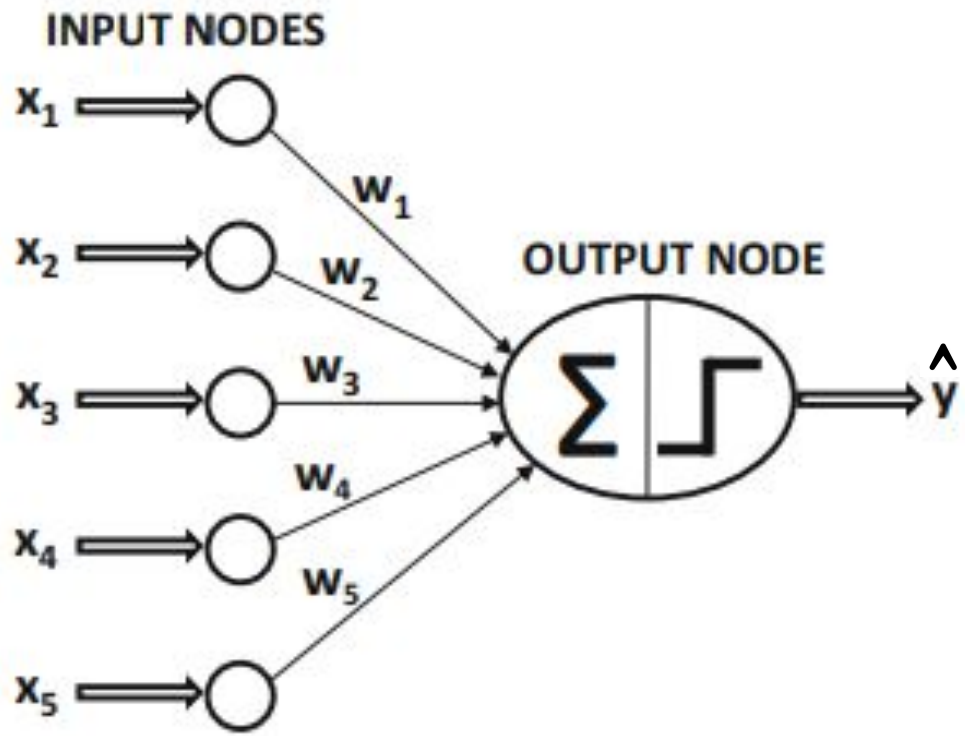
Redes Neuronales Artificiales - ANN

Antes de comprender las ANN debemos entender el modelo más simple de ANN. El perceptrón.



ANN - Perceptrón

Es la arquitectura ANN más simple, desarrollada en 1957 por Frank Rosenblatt.



paper: https://www.ling.upenn.edu/courses/cogs501/Rosenblatt1958.pdf

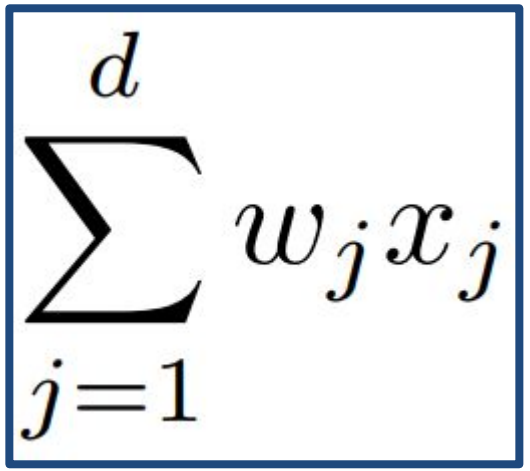
ANN - Perceptrón

Las entradas y salidas son numéricas (x_j).
Cada entrada tiene asociado un peso (w_j).

ANN - Perceptrón

¿Qué hace el nodo de salida? Una operación matemática que se puede desglosar en dos tareas fundamentales en el siguiente orden:

1. La sumatoria de productos de cada entrada con su peso. Es decir = x1.w1 + x2.w2+ . . . + $x_j.w_j$



ANN - Perceptrón

¿Qué hace el nodo de salida? Una operación matemática que se puede desglosar en dos tareas fundamentales en el siguiente orden:

1. La sumatoria de productos de cada entrada con su peso. Es decir = x1.w1 + x2.w2+ . . . + $x_j.w_j$
2. Aplica a la sumatoria, una función de activación. Puede ser cualquiera. Por ejemplo: lineal, heaviside (escalón), signo, etc.

$$\phi\left(\sum_{j=1}^{d} w_j x_j\right)$$

Redes Neuronales Artificiales - ANN

¿Cuál es la salida?

$$\hat{y} = \phi\left(\sum_{j=1}^{d} w_j x_j\right)$$

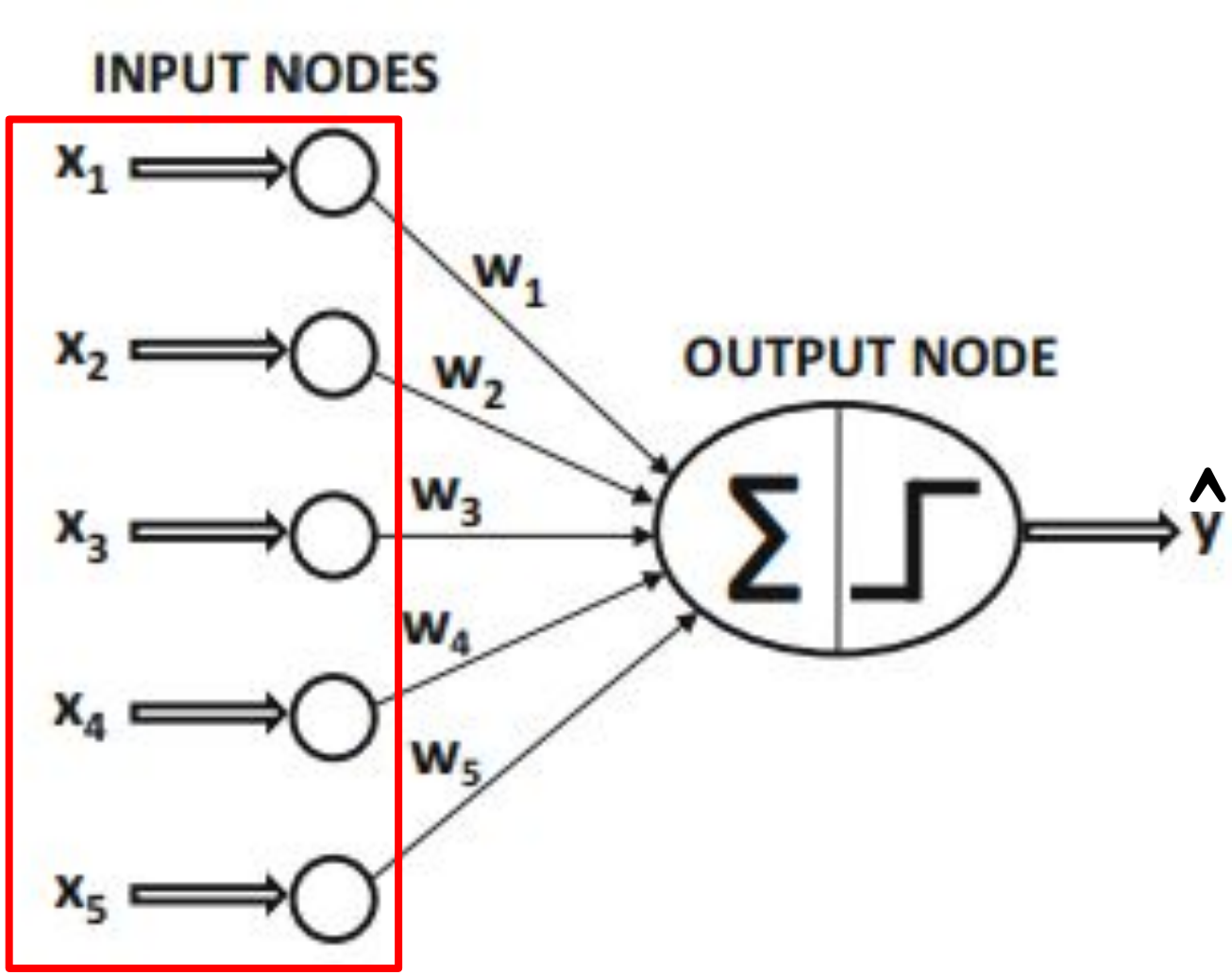
Redes Neuronales Artificiales - ANN

¿Cuál es la salida?

$$\hat{y} = \phi\left(\sum_{j=1}^{d} w_j x_j\right)$$



ANN - Perceptrón

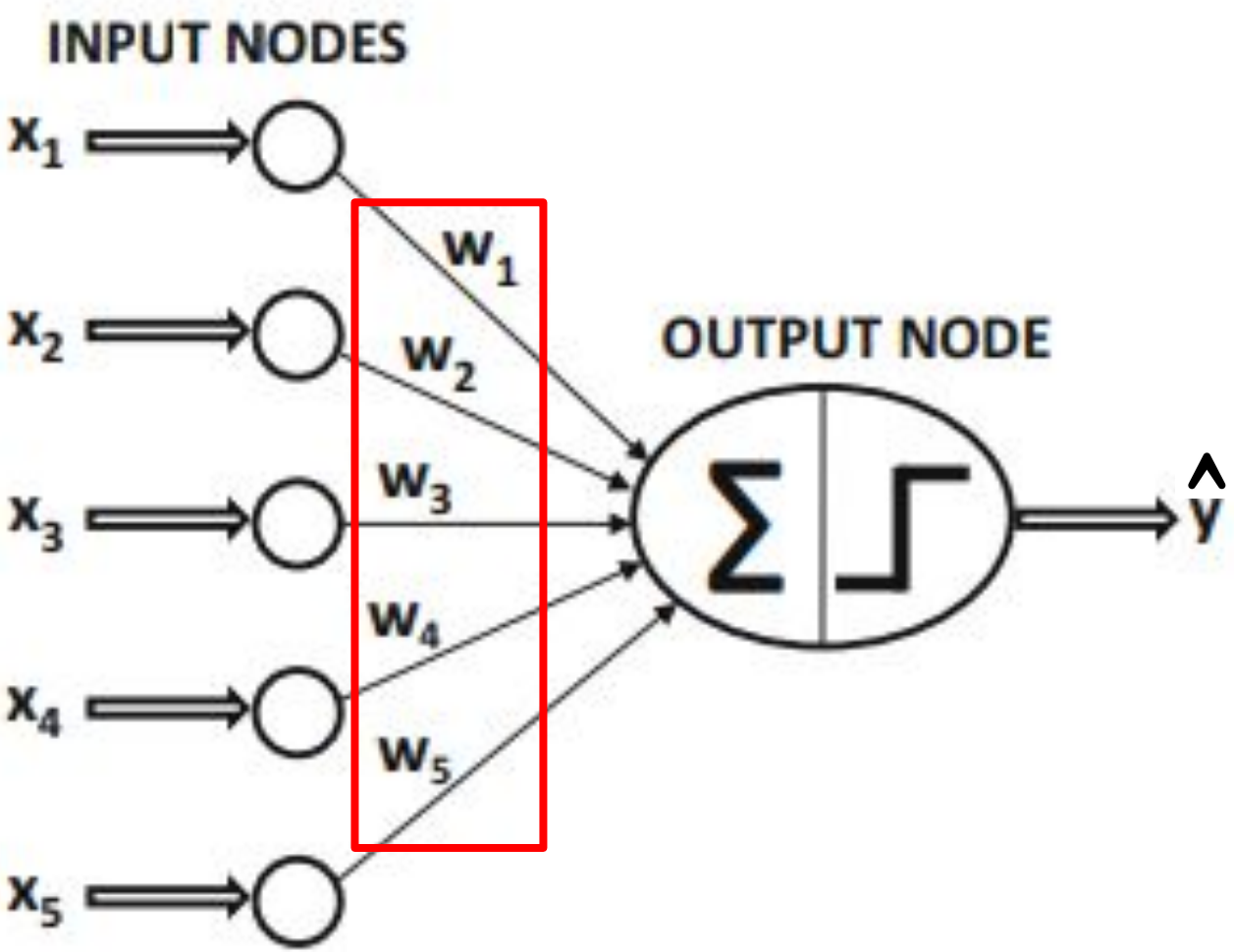


x_1 a x_n → Representan las características que se van a utilizar para realizar la predicción. Por ejemplo. En el dataset iris, corresponde a las dimensiones de la flor.

Un nodo de entrada por cada característica.

Si tenemos que hacer una regresión lineal a partir de dos variables de entrada, tenemos que tener dos nodos de entrada.

ANN - Perceptrón



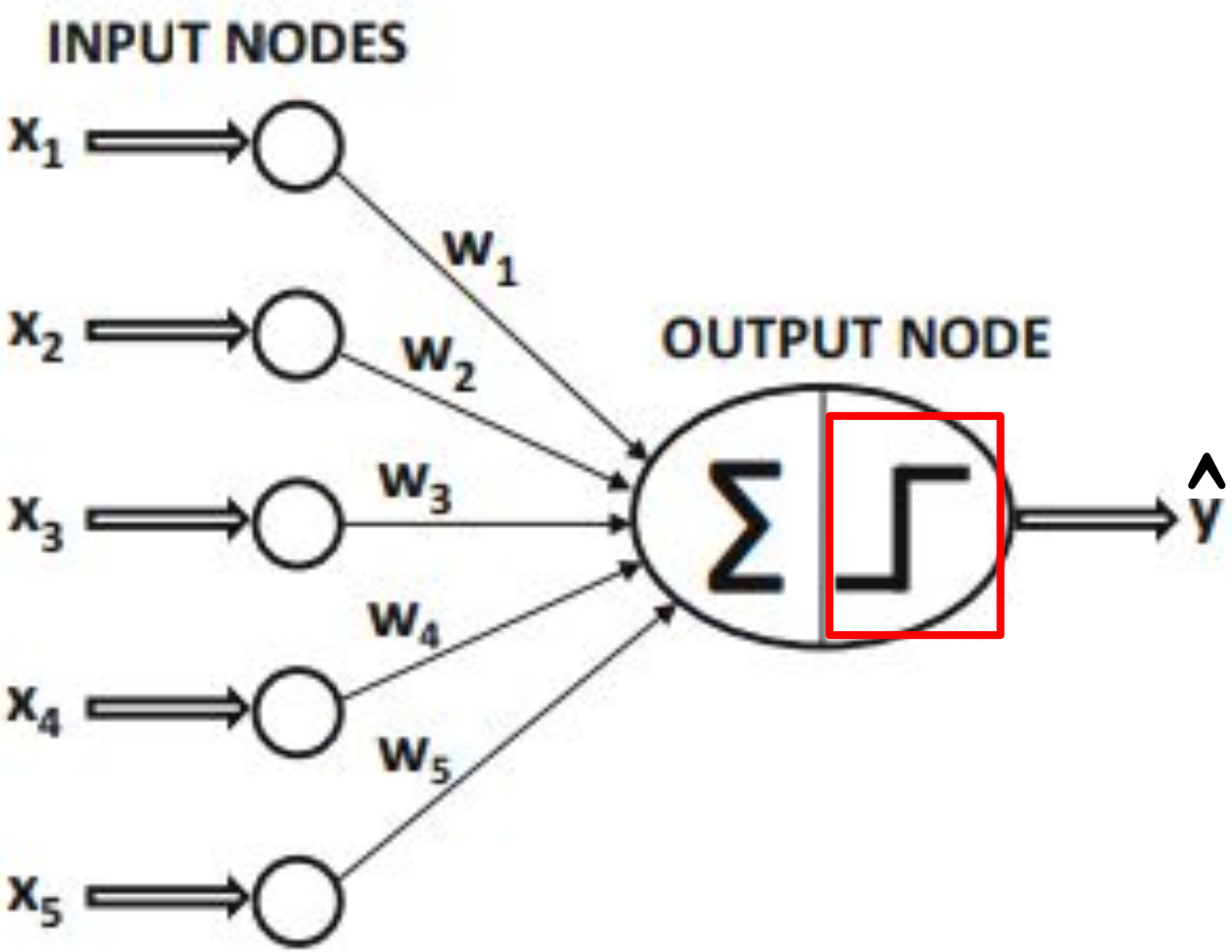
w_1 a w_n → Representan los pesos.

Más adelante ….

Por lo pronto, los pesos se actualizan y ajustan constantemente en la fase de entrenamiento.

El objetivo es encontrar pesos que describan los datos originales.

ANN - Perceptrón



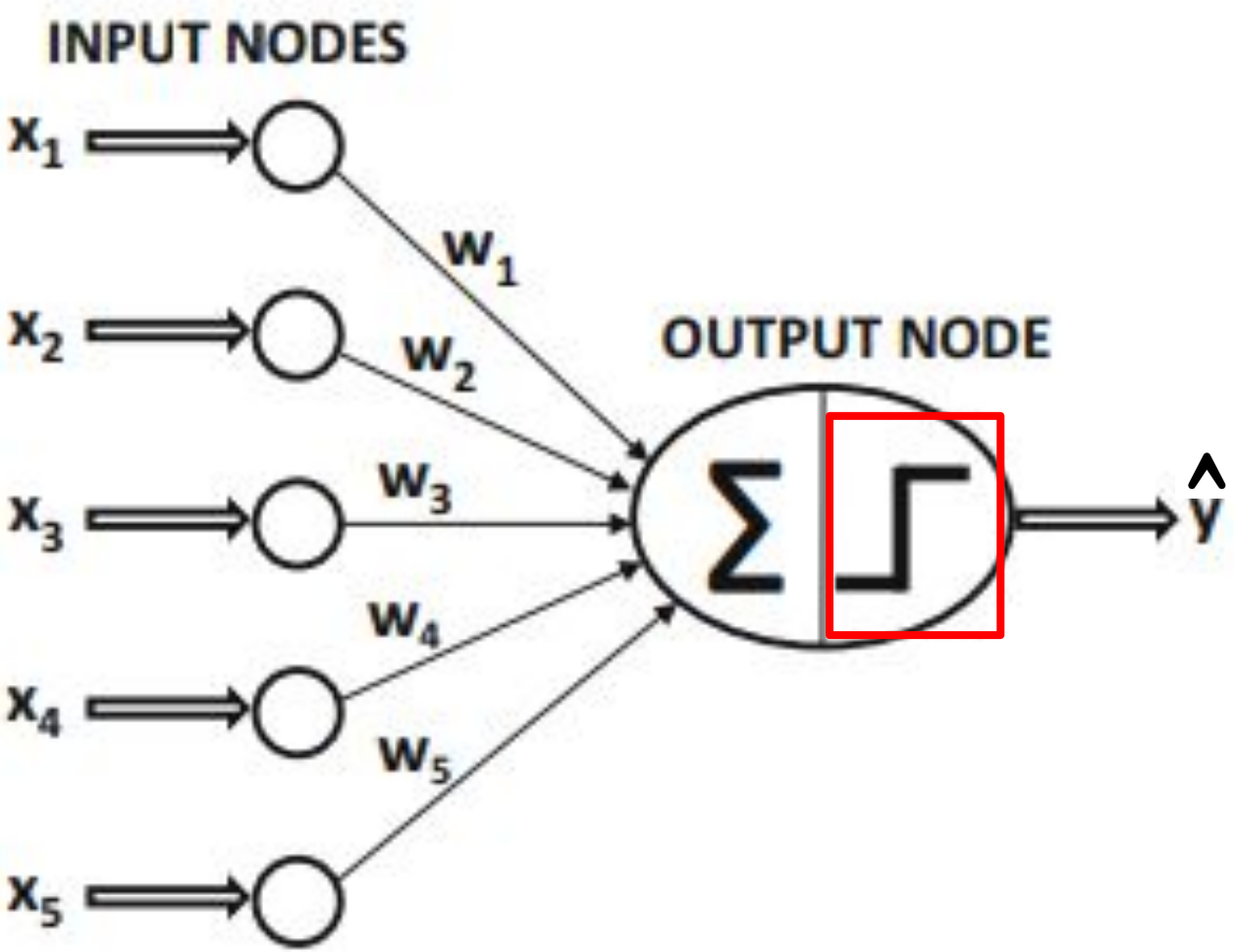
El nodo de salida realiza dos operaciones fundamentales:

1. la sumatoria de productos.
2. la función de activación.

La función de activación permite introducir no linealidades y ayudan al aprendizaje de patrones complejos.

Podemos pensar que realizan un mapeo de los datos de entrada en una salida determinada.

ANN - Perceptrón



¿Qué sucede si no tenemos funciones de activación?

La neurona simplemente devolvería una combinación lineal de la entradas y el aprendizaje sería nulo. Los problemas no son siempre lineales.

ANN - Perceptrón - Funciones de Activación

Sigmoidea

Toma cualquier valor de la entrada y nos devuelve un resultado entre 0 y 1. Es decir, cuanto más positiva es la entrada, más cercano al 1 va a estar la salida. Por otro lado, cuanto más negativa sea la entrada más cercana a 0 va a ser la salida.

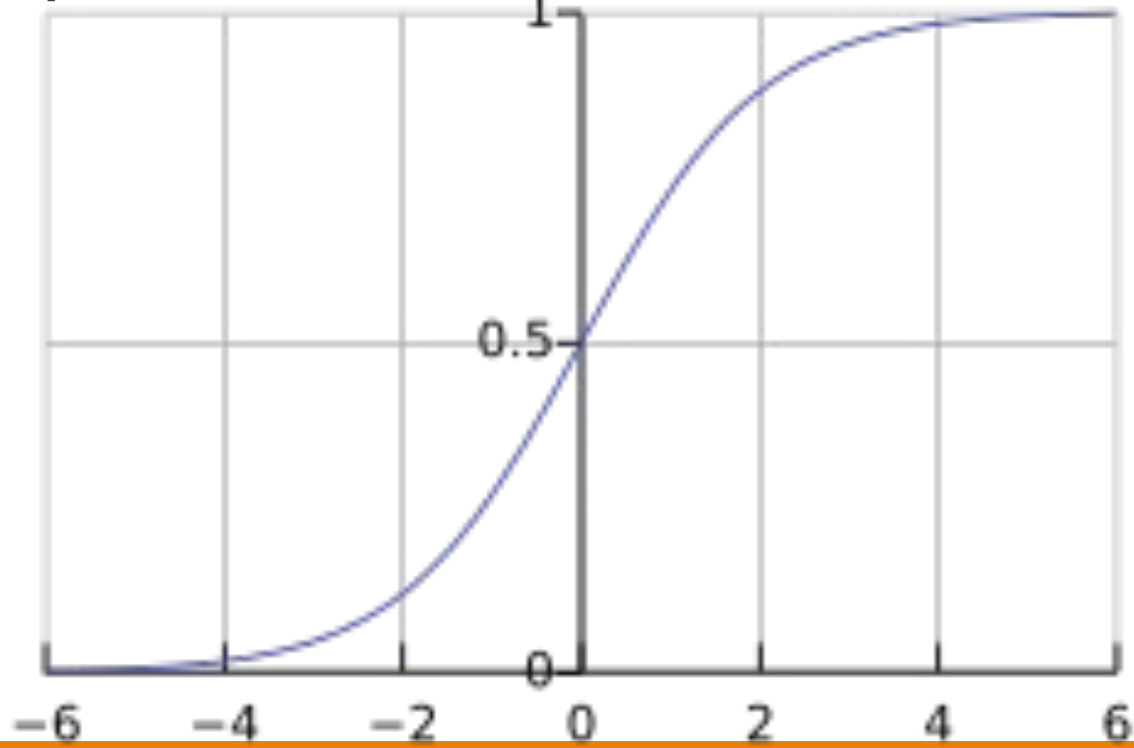


$$f(x) = \frac{1}{1 + e^{-x}}$$

ANN - Perceptrón - Funciones de Activación

Sigmoidea

Ideal para problemas de clasificación binaria. Tenemos un modelo de predicción de mensajes spam. Con la función sigmoidea vamos a obtener un valor entre 0 y 1 en nuestras predicciones.



Si la salida es menor que 0,5 no es spam. Si es mayor o igual que 0,5 es spam.

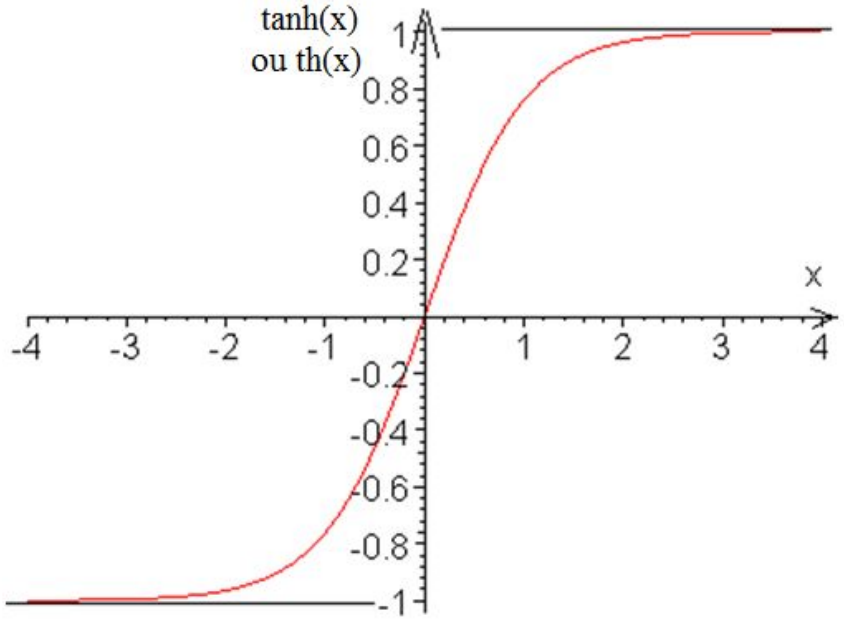
$$f(x) = \frac{1}{1 + e^{-x}}$$

ANN - Perceptrón - Funciones de Activación

Tangente Hiperbólica

Toma cualquier valor de la entrada y nos devuelve un resultado entre -1 y 1. Es decir, cuanto más positiva es la entrada, más cercano al 1 va a estar la salida. Por otro lado, cuanto más negativa sea la entrada más cercana a -1 va a ser la salida. Además si la entrada es más cercana a 0, la salida también va a 0

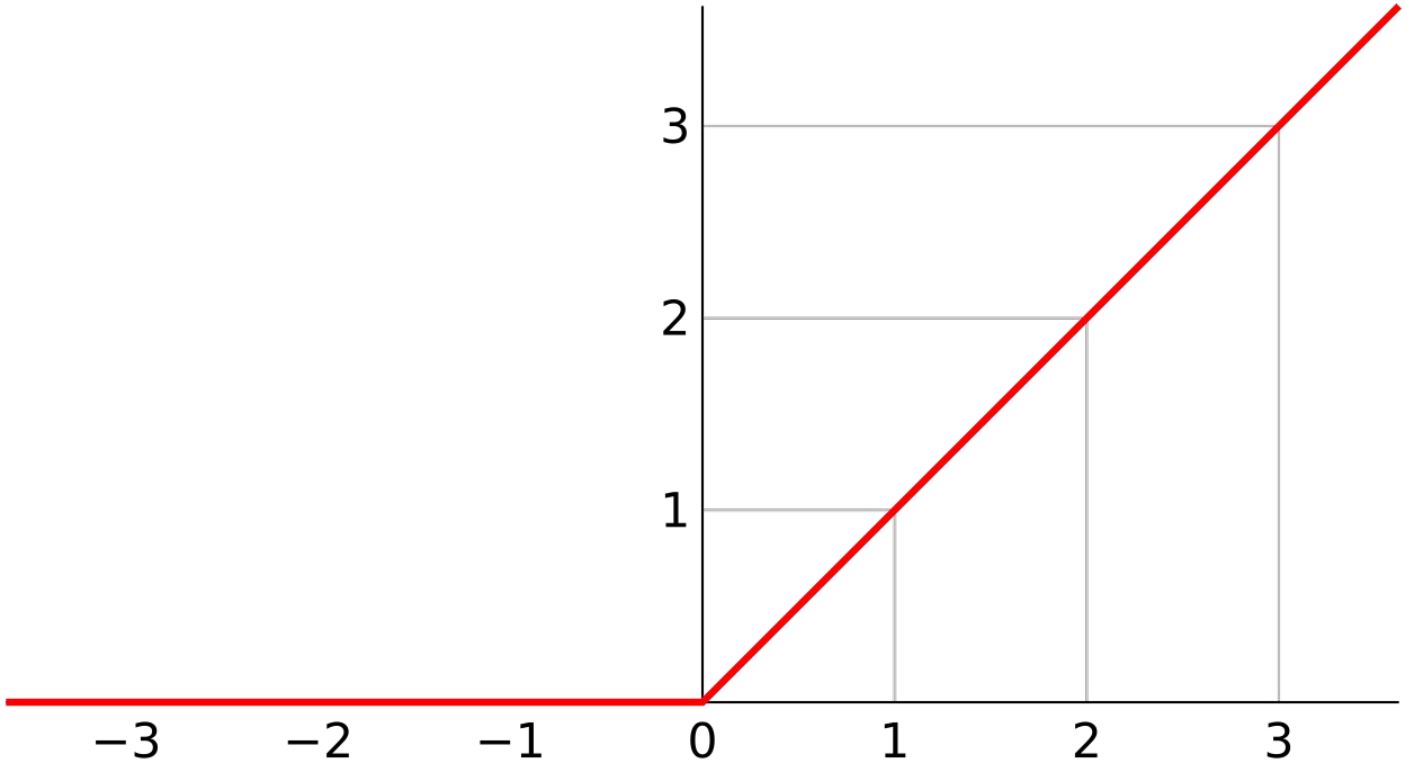
$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



ANN - Perceptrón - Funciones de Activación

ReLu - Rectified Linear Unit.

Si la entrada x es positiva, retrona la entrada. Si la entrada es negativa, retorna 0. De forma simple, ReLu devuelve 0 para cualquier entrada negativa y mantiene la entrada si es positiva.



$$f(x) = \max(0, x)$$

ANN - Perceptrón - Funciones de Activación

SoftMax

Generalmente se utiliza al final de la red neuronal en casos donde se necesita clasificar entradas en múltiples categorías (mas de dos categorias). Toma como entrada un vector de números reales y devuelve una distribución de probabilidades que representa la cercanía a cada categoría/clase.

$$\mathrm{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Útil en problemas de clasificación multiclase. Asegura que las probabilidades de la salida sumen 1. Facilitando la interpretación de los resultados.

ANN - Perceptrón - Funciones de Activación

Resumiendo …

Para problemas de:

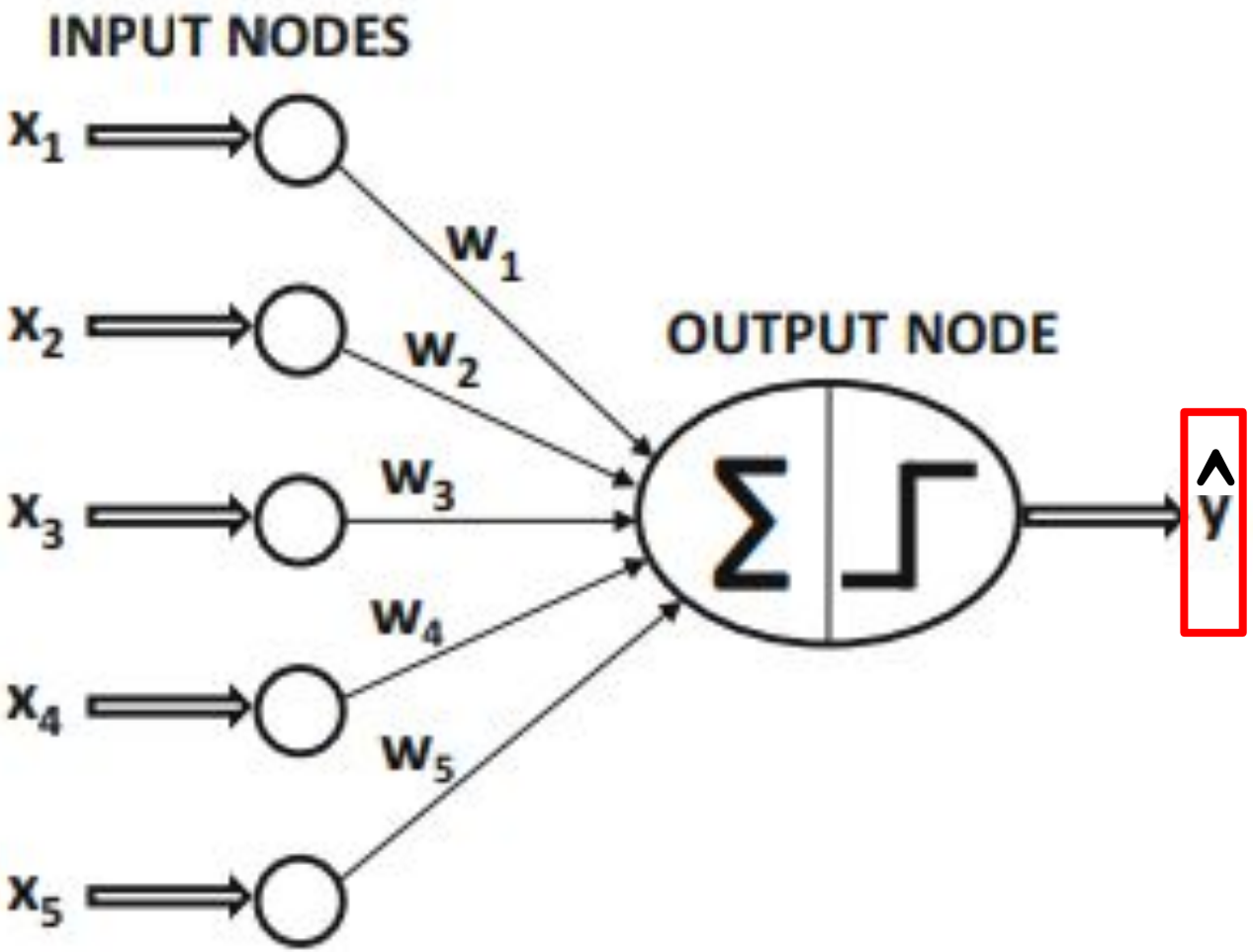
Regresión — Función lineal.

Clasificación Binaria — Función Sigmoidea o Logística.

Clasificación multiclase — Softmax.

En capas intermedias (ocultas) la recomendación es ReLU para CNN.

ANN - Perceptrón



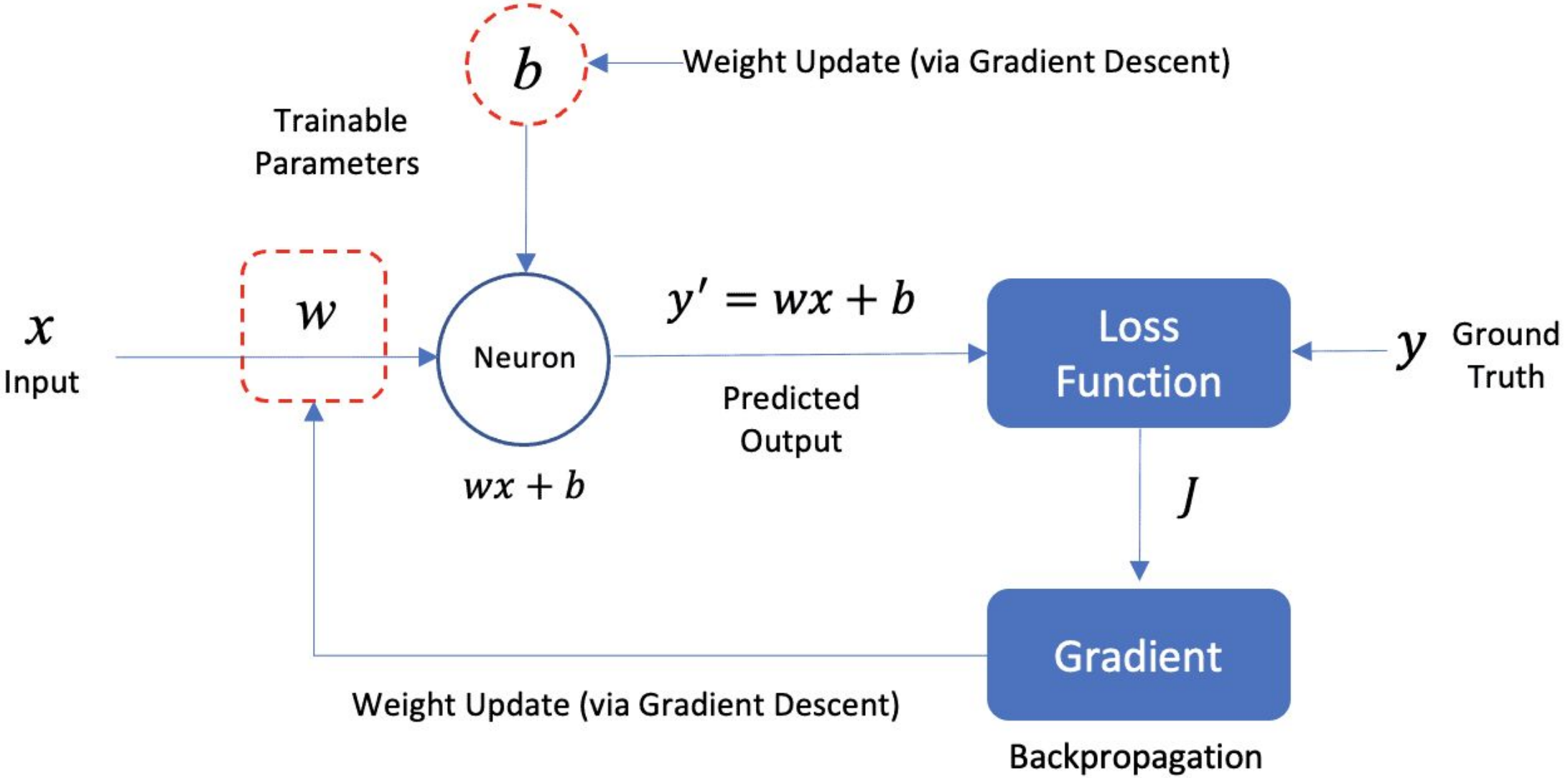
Recapitulando …

Luego de definir la entrada, los pesos y las funciones de activación a la salida de la neurona vamos a tener un valor predecido.

¿Cómo es el proceso de aprendizaje?

Simple, comparamos la salida obtenida con el valor real. Dependiendo de la diferencia, debemos actualizar los pesos para la siguiente iteración.

ANN - Perceptrón



más información en:: https://towardsdatascience.com/deep-learning-illustrated-part-2-how-does-a-neural-network-learn-481f70c1b474/

ANN - Perceptrón - Funciones de Pérdida

Es decir, necesitamos contrastar el valor predecido con el real, a través de una función de pérdida, y a partir de esto utilizar el algoritmo Descenso Gradiente para actualizar los pesos de la neurona.

Existen diferentes Funciones de pérdida (loss functions) y cada una se va a ajustar a un problema determinado.

Más información sobre descenso gradiente:

https://www.ibm.com/es-es/think/topics/gradient-descent

ANN - Perceptrón - Funciones de Pérdida

Regresión:

- Error Cuadrático Medio (MSE)
- Error Cuadrático Medio Logarítmico.
- Error Absoluto Medio.

Clasificación Binaria:

- Entropia Cruzada Binaria.
- Hinge.
- Squared Hinge.

Clasificación multiclase:

- Sparse Multiclass Cross-Entropy Loss
- Kullback Leibler Divergence Loss

Python y Deep Learning

¿Cómo se implementa una neurona?

Al ser un modelo computacional, con cualquier lenguaje de programación.

Python ha ganado terreno en este ámbito y nos ofrece diversas librerías con mucho más que una implementación de una neurona.

Python y Deep Learning

Existen diferentes librerías para trabajar en el contexto de Deep Learning. Algunas de las más conocidas son:

- Tensorflow
- Pytorch
- Keras

Python y Deep Learning

Tensorflow es una plataforma que cubre todo el proceso de diseño y desarrollo de modelos de Machine Learning.

Desarrollada por Google.

Posee un amblio conjunto de bibliotecas y extensiones https://www.tensorflow.org/resources/libraries-extensions?hl=es-419



Python y Deep Learning

¿Por qué usar Tensorflow?

- Tiene soporte para lenguajes como Python, R, C++ y Go.
- Integración con otras librerías para complementar las capacidades de Tensorflow (scikit-learn, caffe, pickle, keras, pandas, numpy).
- De código abierto y gran soporte de la comunidad.
- Entrenamiento de modelos con soporte para CPU, GPU y TPU.

Python y Deep Learning

PyTorch es un framework de machine learning que facilita y acelera el proceso que va desde el diseño del prototipo hasta la puesta en producción de modelo de Machine Learning

Diseñado bajo la filosofía Python lo que permite una arraigada integración con otras librerías de Python.



Python y Deep Learning

¿Por qué usar pytorch?

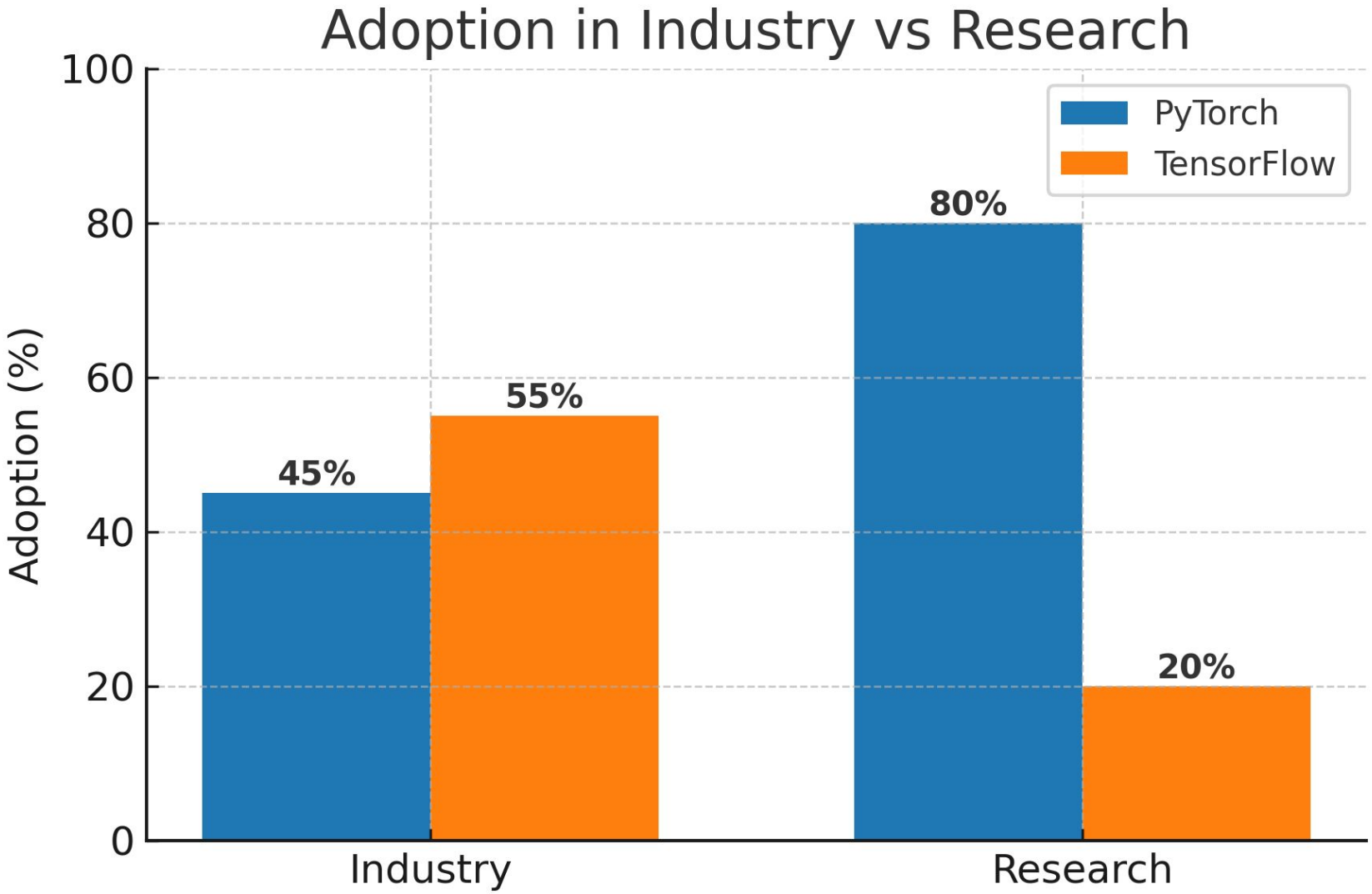


Python y Deep Learning

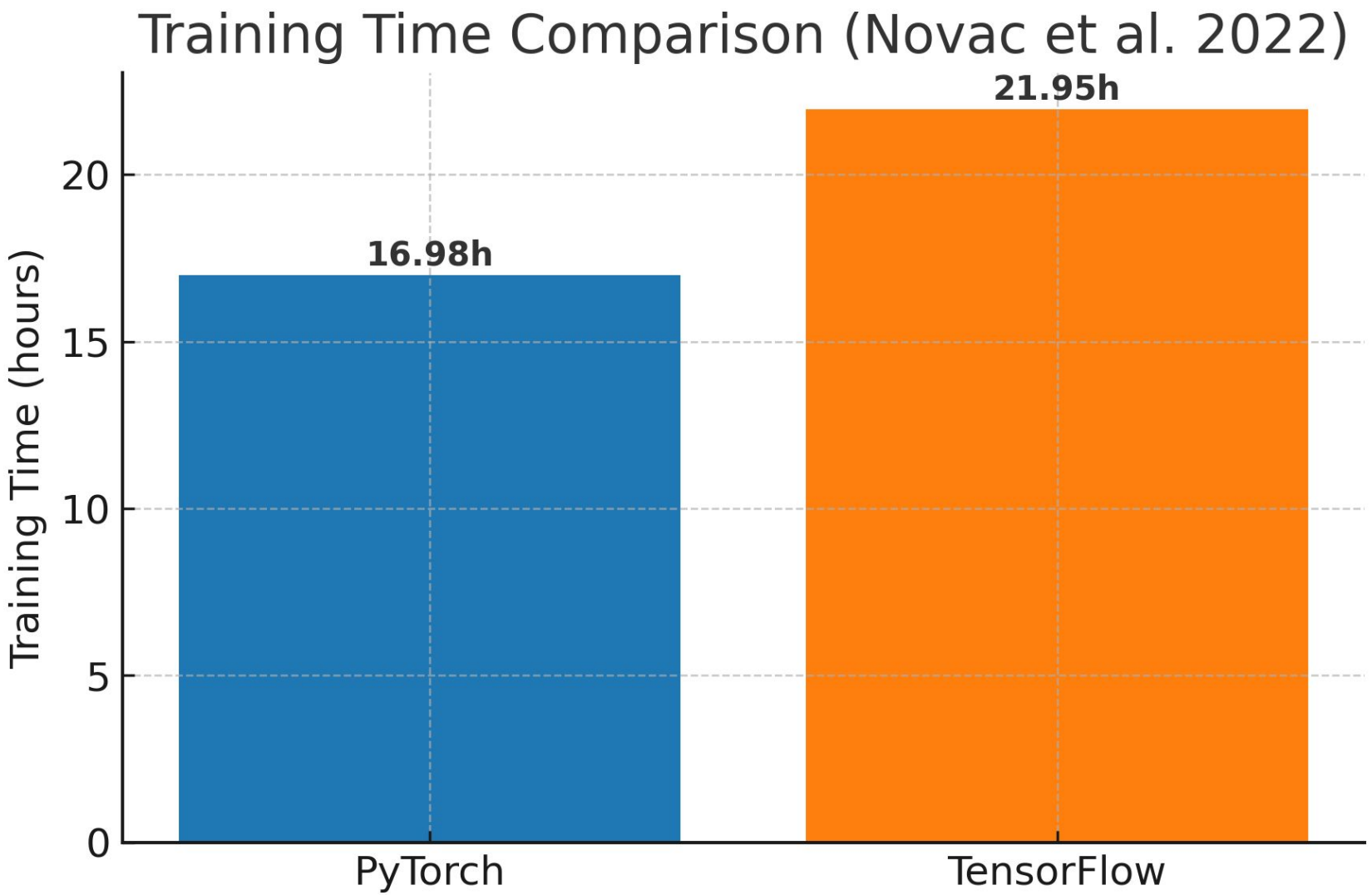
¡Tensorflow vs Pytorch!

https://arxiv.org/html/2508.04035v1
https://ieeexplore.ieee.org/abstract/document/9515098
https://www.mdpi.com/1424-8220/22/22/8872

Python y Deep Learning



Python y Deep Learning



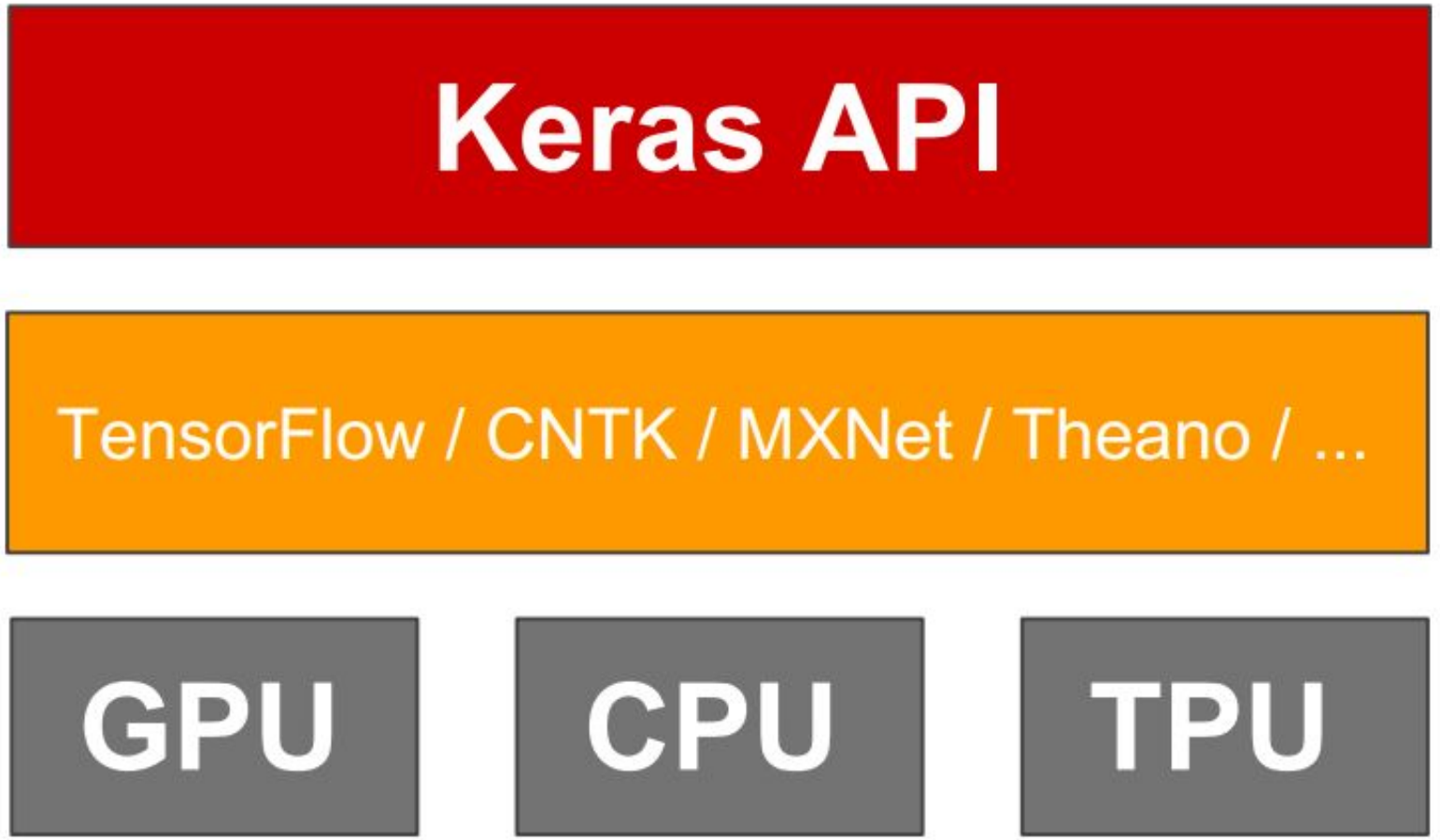
Python y Deep Learning

¿Y Keras?

Formalmente una librería de alto nivel que facilita la codificación, entrenamiento y validación de modelos de machine learning.

Lo clave en Keras es el alto nivel de abstracción. Es decir, se puede implementar redes neuronales de forma muy simple y con pocas líneas de código.

Python y Deep Learning



Python y Deep Learning

```python
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(num_features,)),
    keras.layers.Dense(512, activation="relu"),
    keras.layers.Dense(512, activation="relu"),
    keras.layers.Dense(num_classes, activation="softmax"),
])
model.summary()

model.compile(
    optimizer=keras.optimizers.AdamW(learning_rate=1e-3),
    loss=keras.losses.CategoricalCrossentropy(),
    metrics=[
        keras.metrics.CategoricalAccuracy(),
        keras.metrics.AUC(),
    ],
)

history = model.fit(
    x_train, y_train, batch_size=64, epochs=8, validation_split=0.2
)
evaluation_scores = model.evaluate(x_val, y_val, return_dict=True)
predictions = model.predict(x_test)
```



```
$ python example.py
Using TensorFlow backend
```



```
$ python example.py
Using PyTorch backend
```



```
$ python example.py
Using JAX backend
```

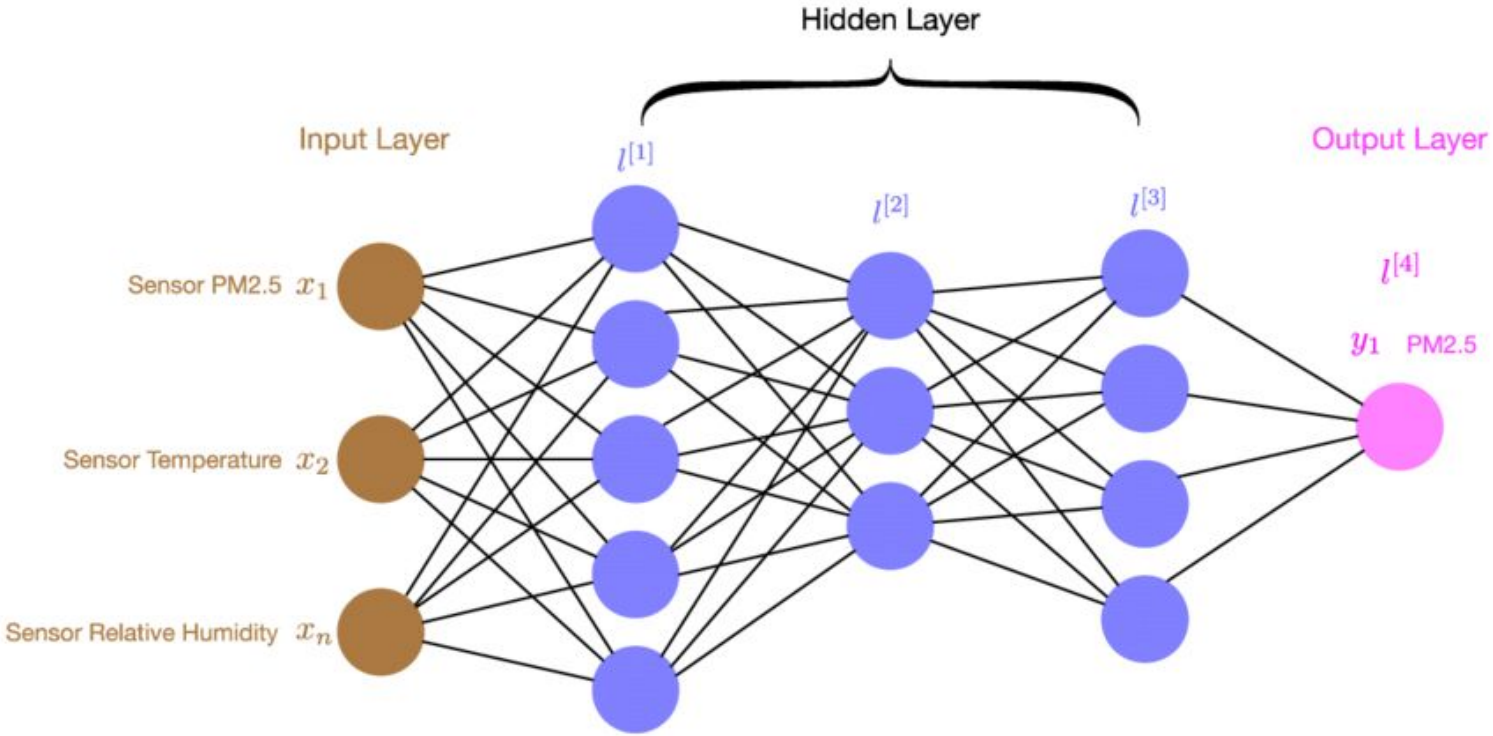
Ejemplos!!!



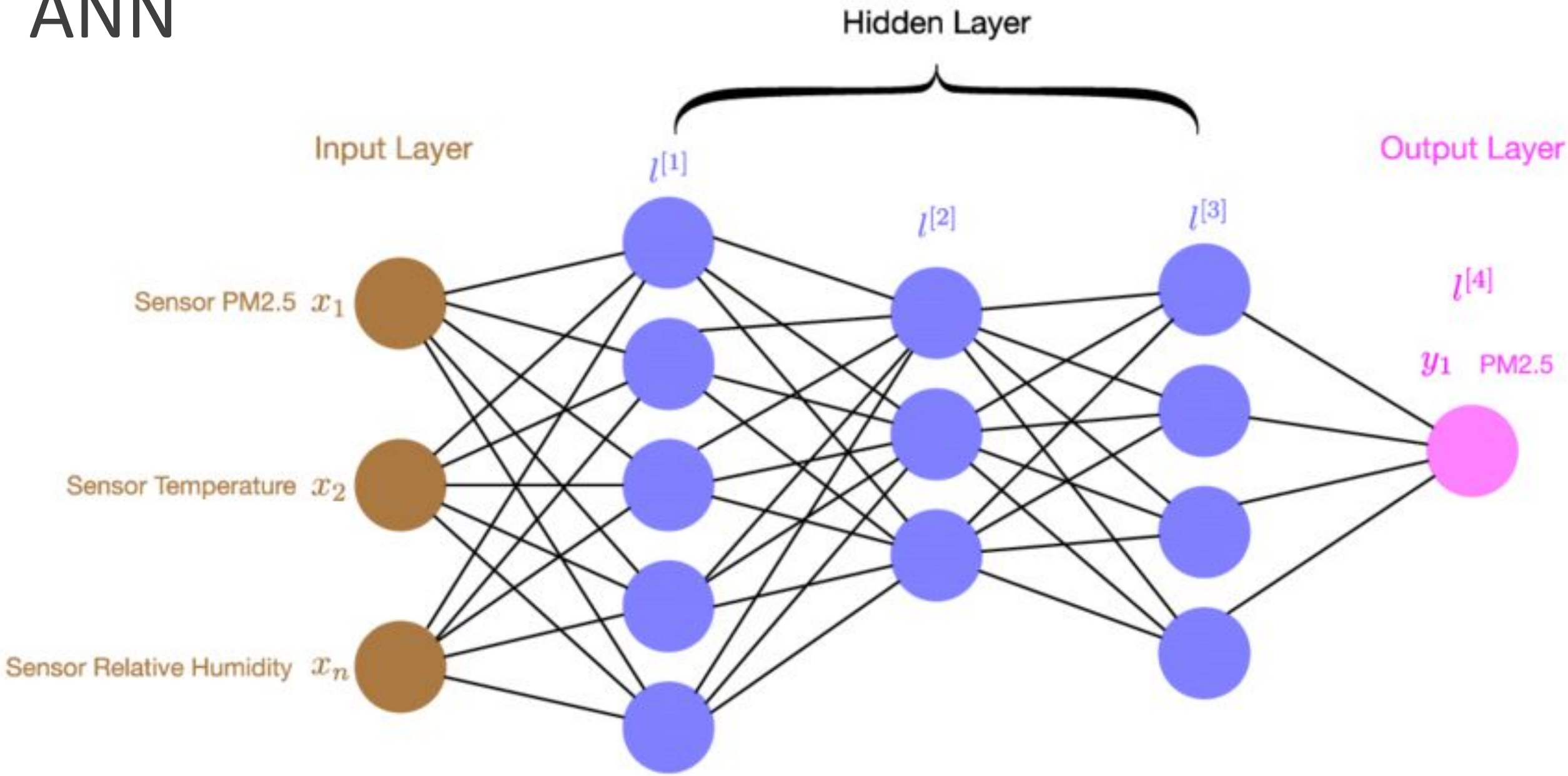
ANN

De los ejemplos vimos que con una simple neurona no es suficiente. No podemos “aprender” con certeza.

Para lograr mayor precisión, debemos combinar diferentes neuronas, organizadas en capas, y conectar capas de neuronas entre sí.

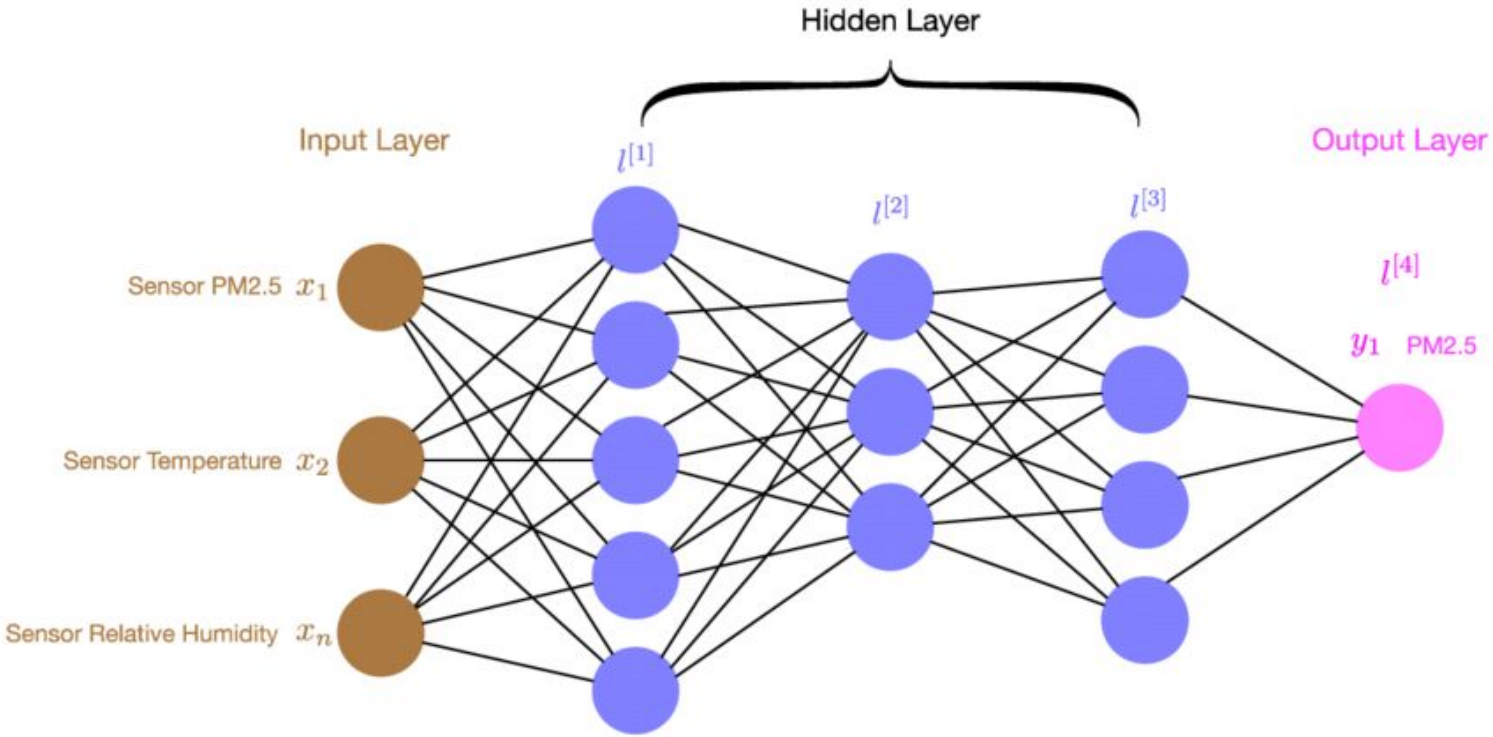


ANN


Hidden Layer
Input Layer
Output Layer
$l^{[1]}$
$l^{[2]}$
$l^{[3]}$
$l^{[4]}$
Sensor PM2.5 x_1
Sensor Temperature x_2
Sensor Relative Humidity x_n
y_1 PM2.5

ANN

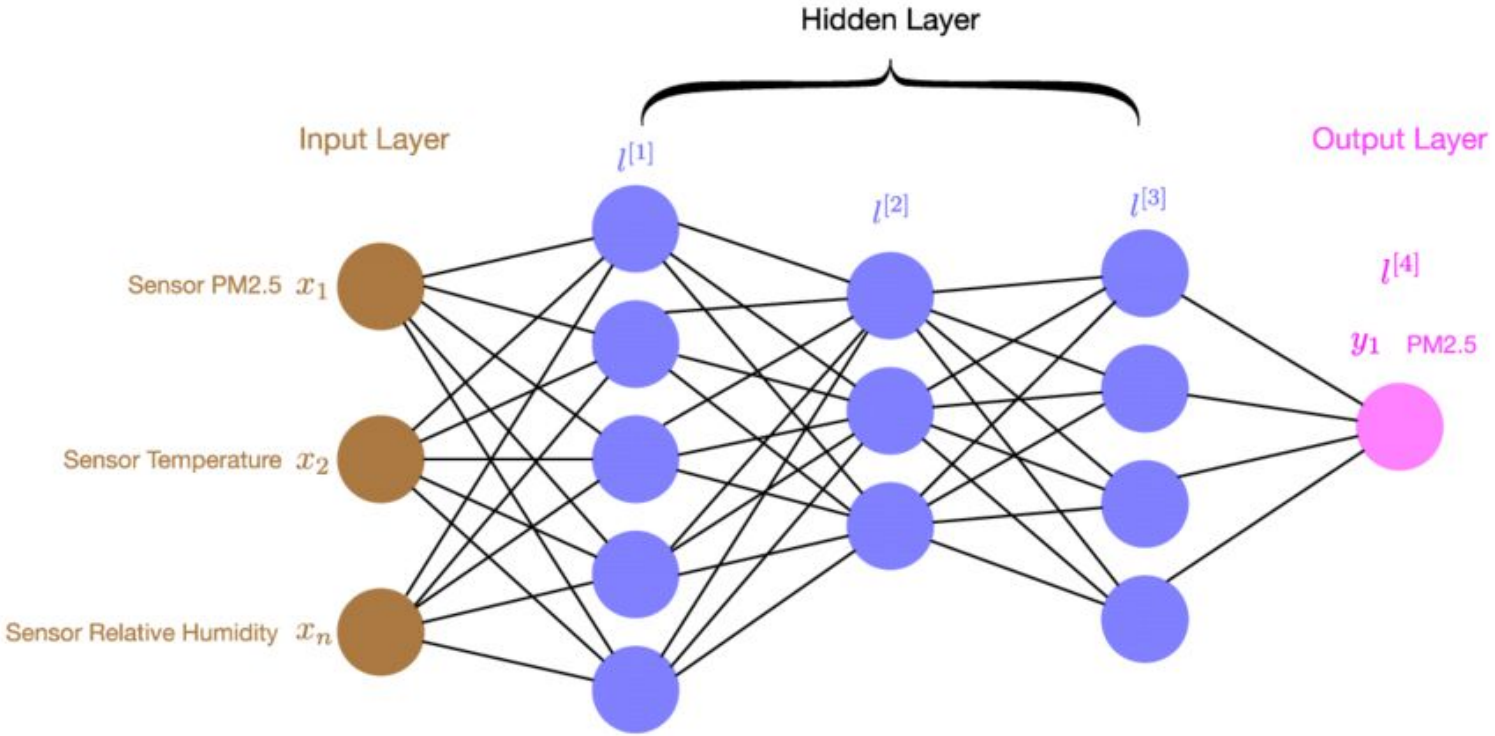
Para agregar más neuronas a una capa, hay que cambiar el primer parámetro (units). Si se desea agregar más capas, es tan simple como agregar un nuevo elemento al modelo secuencial.



ANN

Agregando más neuronas a una capa:

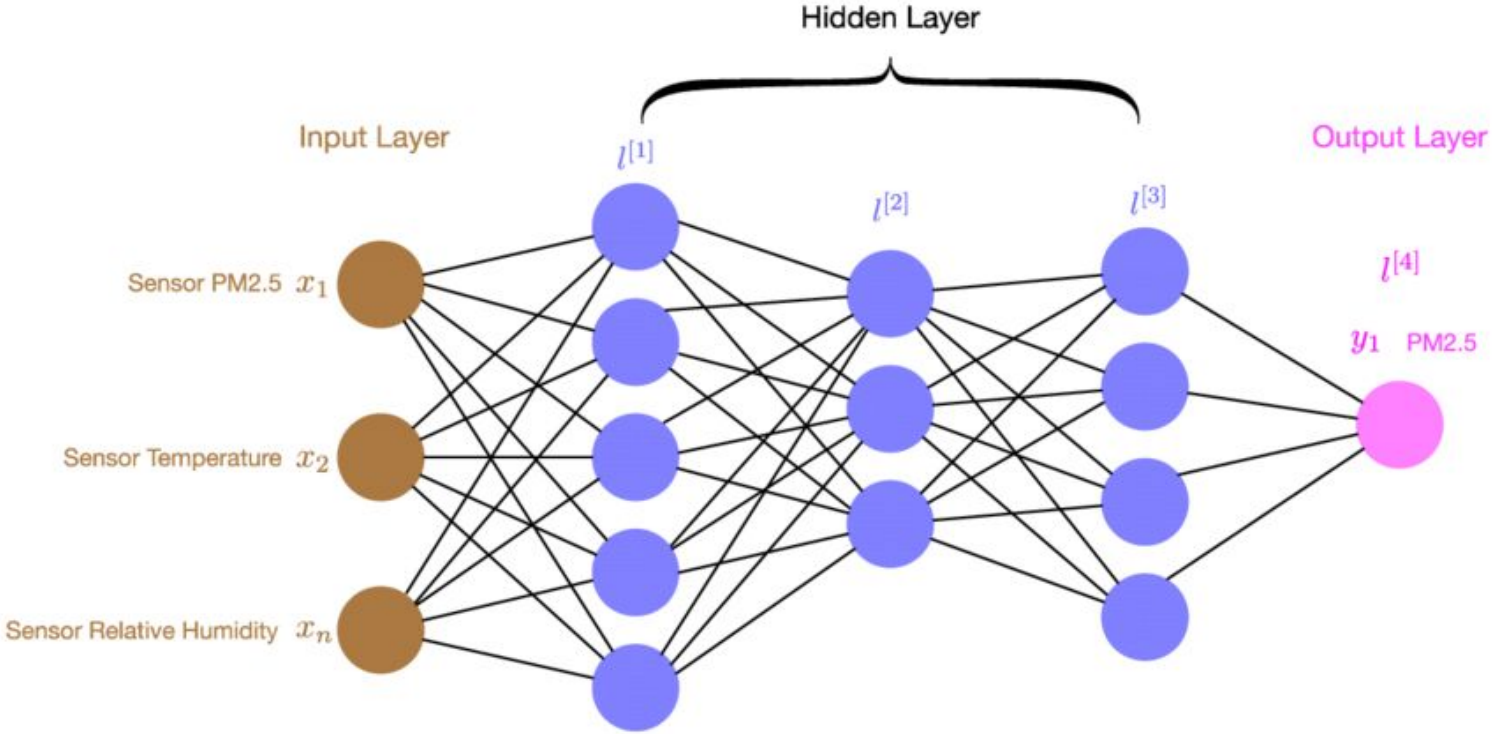
```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(6, activation="relu", input_shape=(3,))
])
```



ANN

Agregando más capas a una red:

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(6, activation="relu", input_shape=(3,)),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
```



ANN

¿Cómo sé la cantidad de neuronas por capa y la cantidad de capas?

¿Cómo determinar la función de activación de cada capa?

¿Cómo averiguar la cantidad de neuronas de la capa de entrada y de salida?

¿Cómo saber la cantidad de capas oculta con la respectiva cantidad de neuronas?



ANN

¿Cómo sé la cantidad de neuronas por capa y la cantidad de capas?

¿Cómo determinar la función de activación de cada capa?

¿Cómo averiguar la cantidad de neuronas de la capa de entrada y de salida?

¿Cómo saber la cantidad de capas oculta con la respectiva cantidad de neuronas?

En principio no hay reglas definidas. Todo es posible. Lo que nos termina limitando y condicionando son los datos.

ANN

¿Cómo averiguar la cantidad de neuronas de la capa de entrada y de salida? Algunas “reglas”:

La cantidad de neuronas de la capa de entrada queda determinada por la cantidad de características elegidas, de nuestro dataset, para realizar la predicción.

De la misma forma, la cantidad de neuronas en la salida está relacionada con los datos a estimar. Si tenemos que determinar cuál de las 10 categorías pertenece una imagen, la cantidad de neuronas en la salida es 10 (softmax). Para problemas de clasificación binaria, la cantidad de neuronas de salida es 1 (sigmoidea).

ANN

¿Cómo saber la cantidad de capas oculta con la respectiva cantidad de neuronas? Algunas “reglas”:

Dos opciones. La primera:

- Agregar capas sucesivas y al mismo tiempo decrementar la cantidad de neuronas. Pirámide. Por ejemplo:
  - Dense(64, activation=”relu”, input_shape=(16,)
  - Dense(32, activation=”relu”)
  - Dense(8, activation=”relu”)
  - Dense(1, activation=”sigmoid”)

ANN

¿Cómo saber la cantidad de capas oculta con la respectiva cantidad de neuronas? Algunas “reglas”:

Dos opciones. La segunda:

- Agregar capas sucesivas y mantener la cantidad de neuronas hasta la capa de salida. Por ejemplo:
  - Dense(32, activation=”relu”, input_shape=(16,)
  - Dense(32, activation=”relu”)
  - Dense(32, activation=”relu”)
  - Dense(1, activation=”sigmoid”)

La recomendación en ambos casos es siempre elegir como función de activación ReLU o sus variantes.

ANN

Existe una tercera opción que implica estimar hiperparámetros.

¿Qué son los hiperparámetros?

Variables de configuración que los científicos de datos establecen de antemano para gestionar el proceso de entrenamiento de un modelo de aprendizaje automático.

Encontrar la combinación adecuada de hiperparámetros es esencial para obtener el mejor rendimiento de los modelos de aprendizaje supervisado y no supervisado.

Algunos hiperparámetros: learning rate, cantidad de capas ocultas, cantidad de neuronas, función de activación por capa, número de épocas, tamaño de lote.

fuente: https://www.ibm.com/es-es/think/topics/hyperparameter-tuning?mhsrc=ibmsearch_a&mhq=hiperparametros

ANN

En keras, para estimar hiperparámetros:

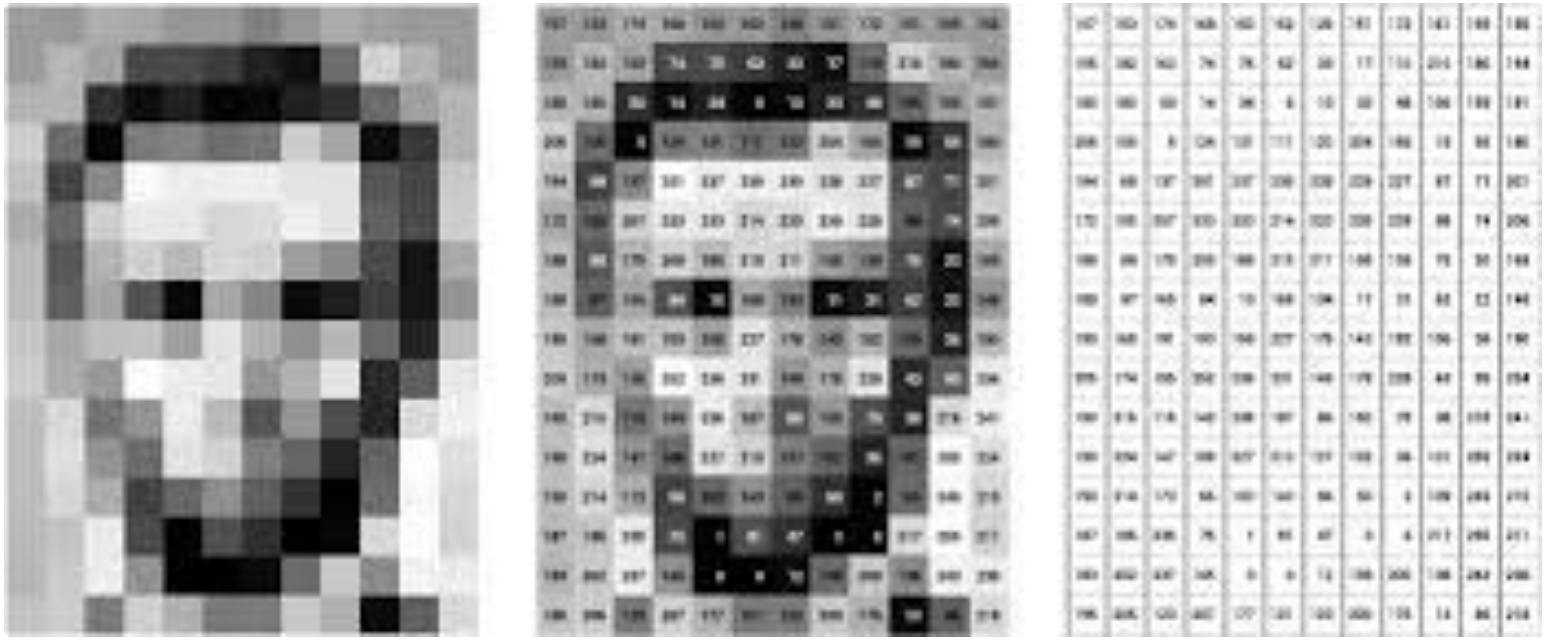
https://keras.io/keras_tuner/getting_started/

pip install keras-tuner

CNN - Redes Neuronales Convolucionales

Existen otro tipo de arquitectura de redes neuronales que nos permiten aprender a partir de datos como imágenes, audios y videos. Formalmente para trabajar con datos en forma de “rejilla”.

Por ejemplo, una imagen de 64x64 está conformado por 4096 pixeles. Cada pixel tiene una posición en la grilla.



CNN - Redes Neuronales Convolucionales

Entonces, ¿qué es una convolución?

La convolución es una operación matemática que combina dos funciones para describir la superposición entre ambas. La convolución toma dos funciones, “desliza” una sobre la otra, multiplica los valores de las funciones en todos los puntos de superposición, y suma los productos para crear una nueva función. Este proceso crea una nueva función que representa cómo interactúan las dos funciones originales entre sí.

CNN - Redes Neuronales Convolucionales

Entonces, ¿qué es una convolución?

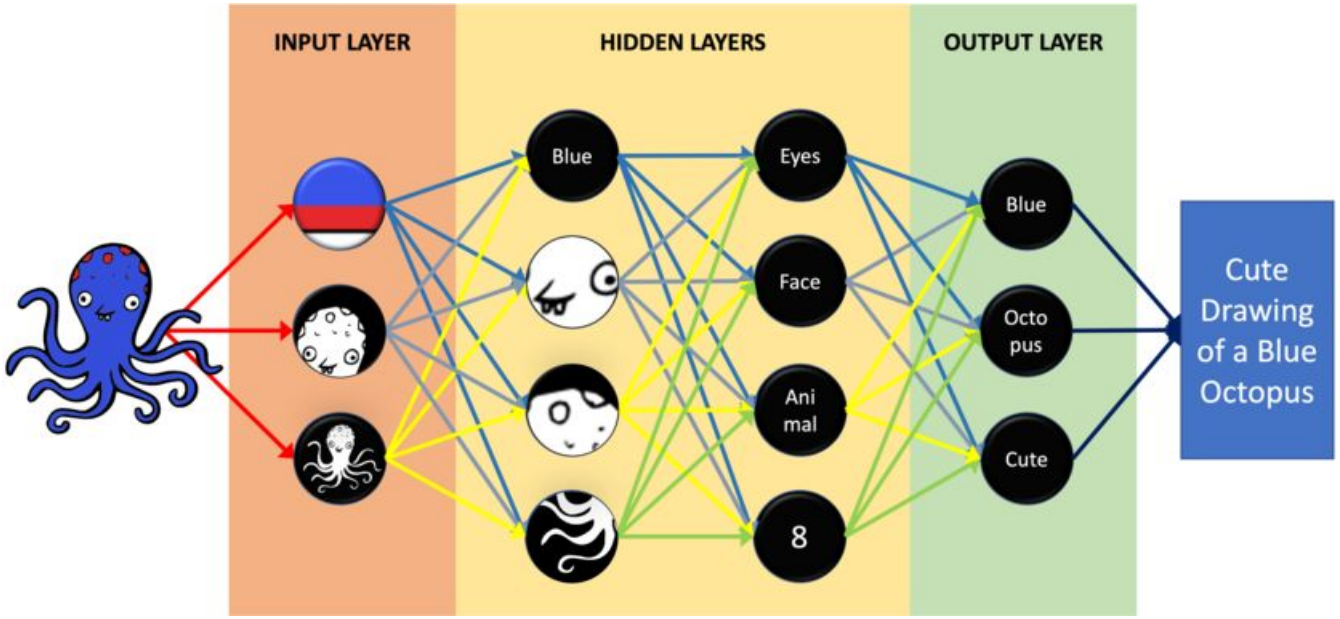
Formalmente, la convolución es una integral que expresa la cantidad de superposición de una función f(t) cuando se desplaza sobre otra función g(t)de este modo:

$$(f * g)(t) \approx^{def} \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

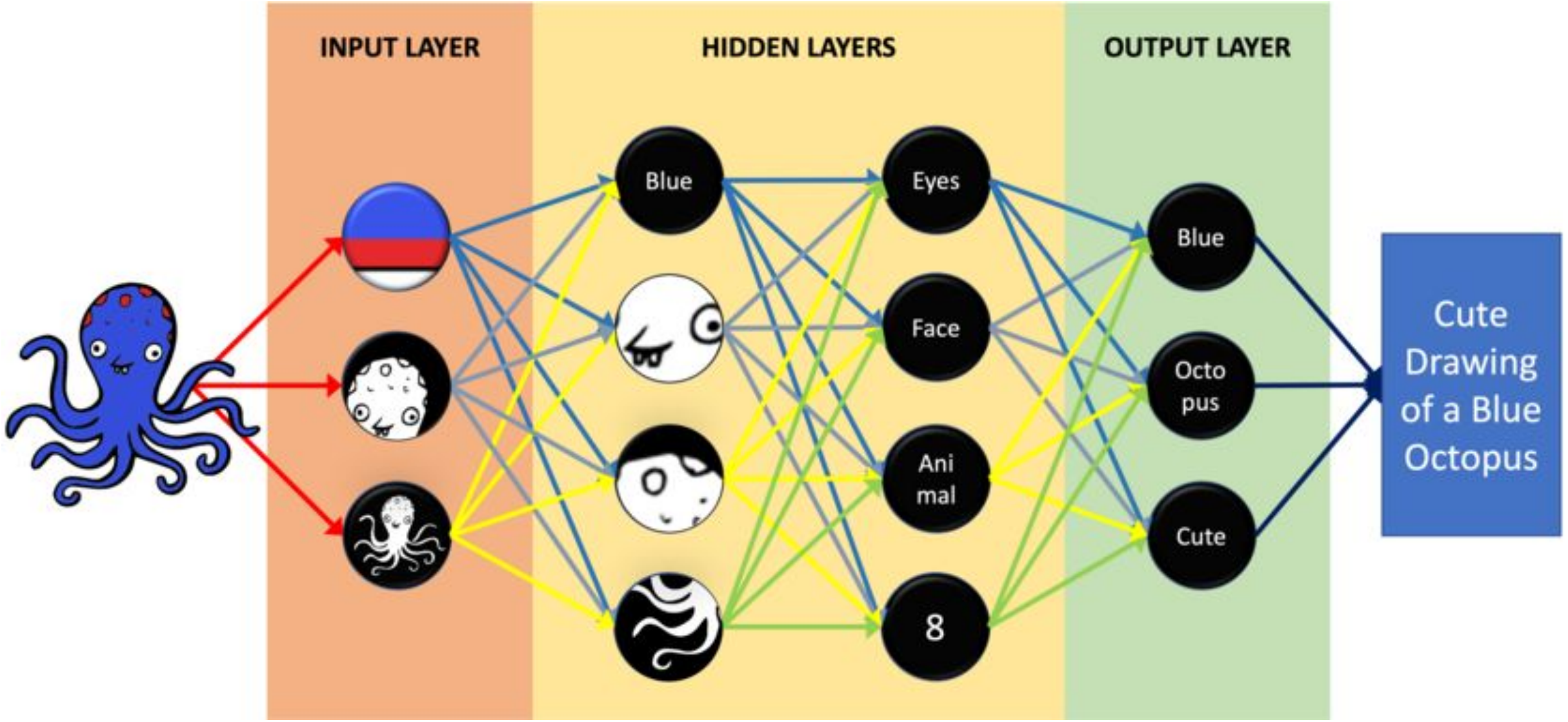
fuente: https://la.mathworks.com/discovery/convolution.html

CNN - Redes Neuronales Convolucionales

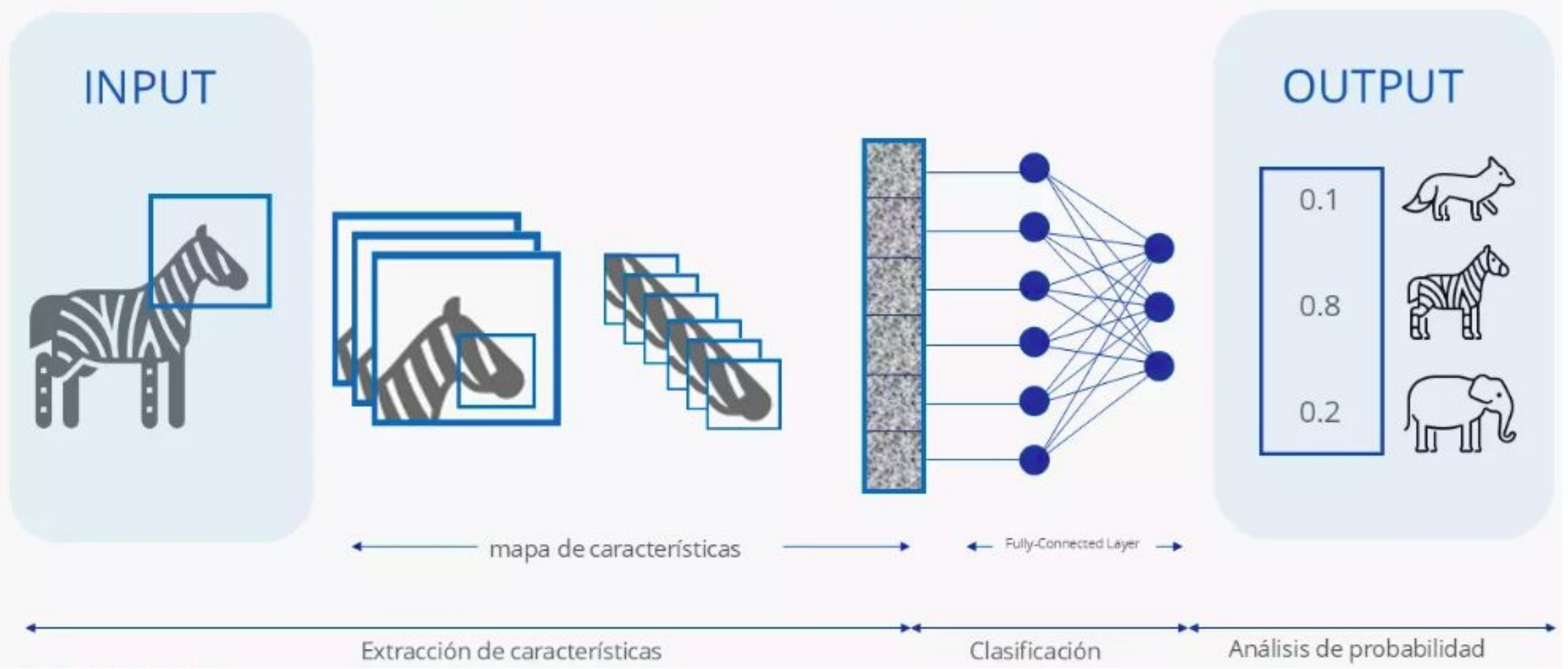
En una CNN las convoluciones son fundamentales para caracterizar los mejor posible una imagen, por ejemplo. Es decir, la convolución nos va a servir para extraer las características que mejor representen la imagen de entrada. No perdamos de vista el enfoque de aprendizaje.



CNN - Redes Neuronales Convolucionales



CNN - Redes Neuronales Convolucionales



Ejemplos!!!



CNN - Redes Neuronales Convolucionales

Las CNN están conformadas por 3 capas principales:

- Capa de Convolución.
- Capa de Pulido.
- Capa Totalmente Conectada.

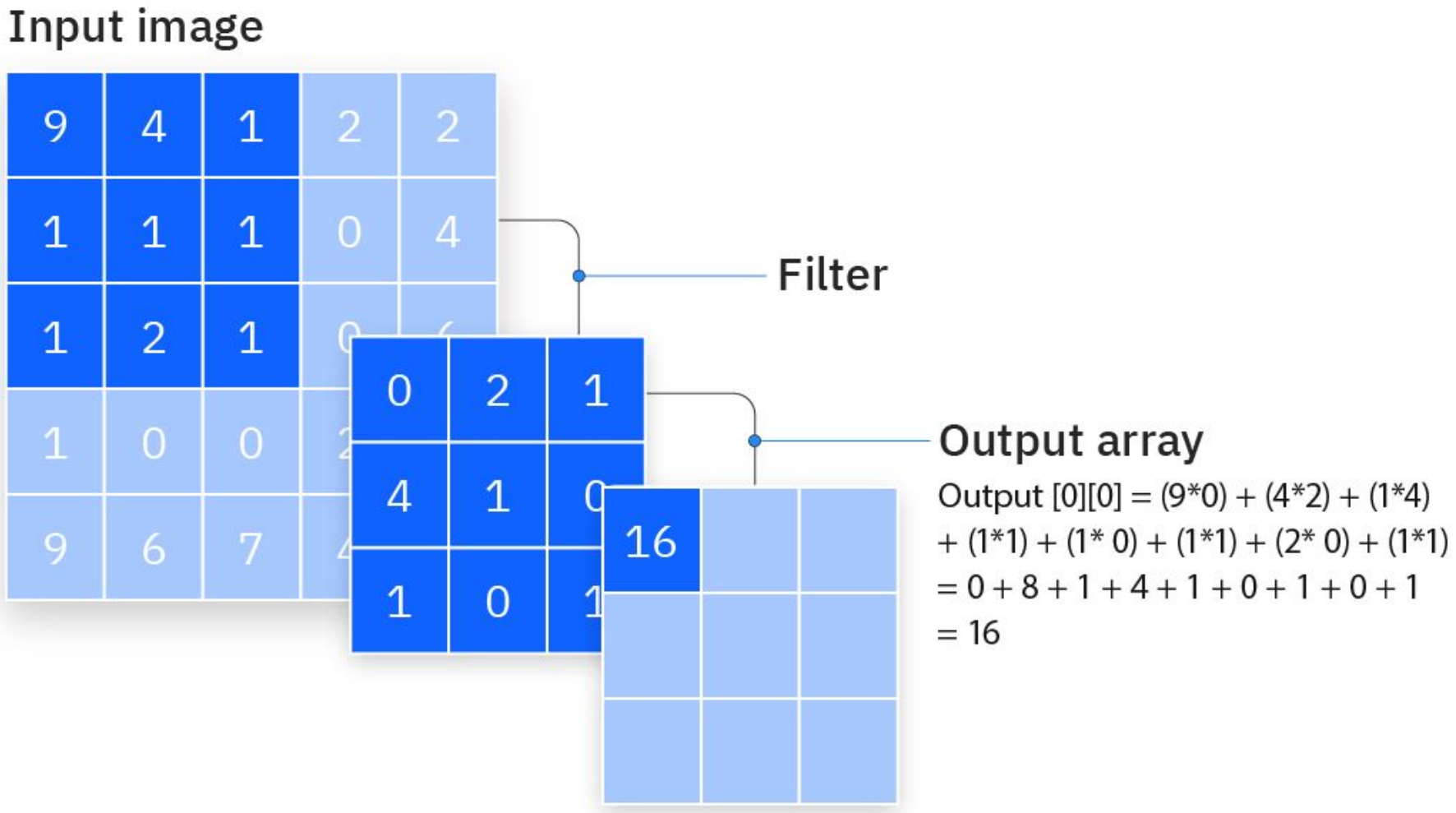
CNN - Redes Neuronales Convolucionales

La capa de convolución siempre es la primera en una CNN y la más importante. En esta capa se extraen las características más representativas de las imágenes.

Esta capa, como mínimo, va a necesitar 3 parámetros:

- La imagen de entrada (en formato tensor).
- Un filtro o kernel.
- Y la cantidad de mapas de características a trabajar: un vector con los pesos de lo más representativo de la imagen, luego de aplicar el filtro de convolución.

CNN - Redes Neuronales Convolucionales

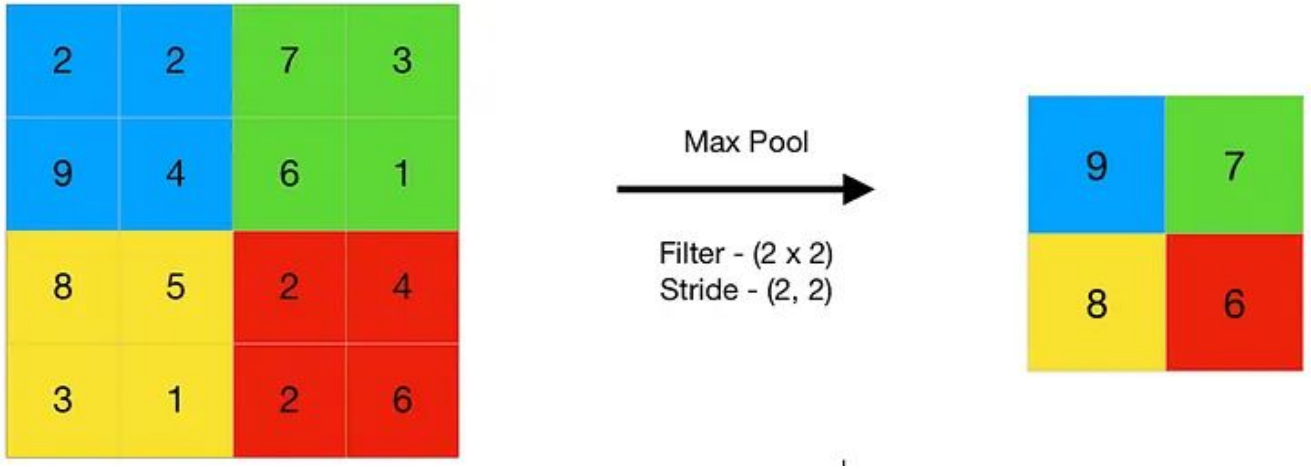


CNN - Redes Neuronales Convolucionales

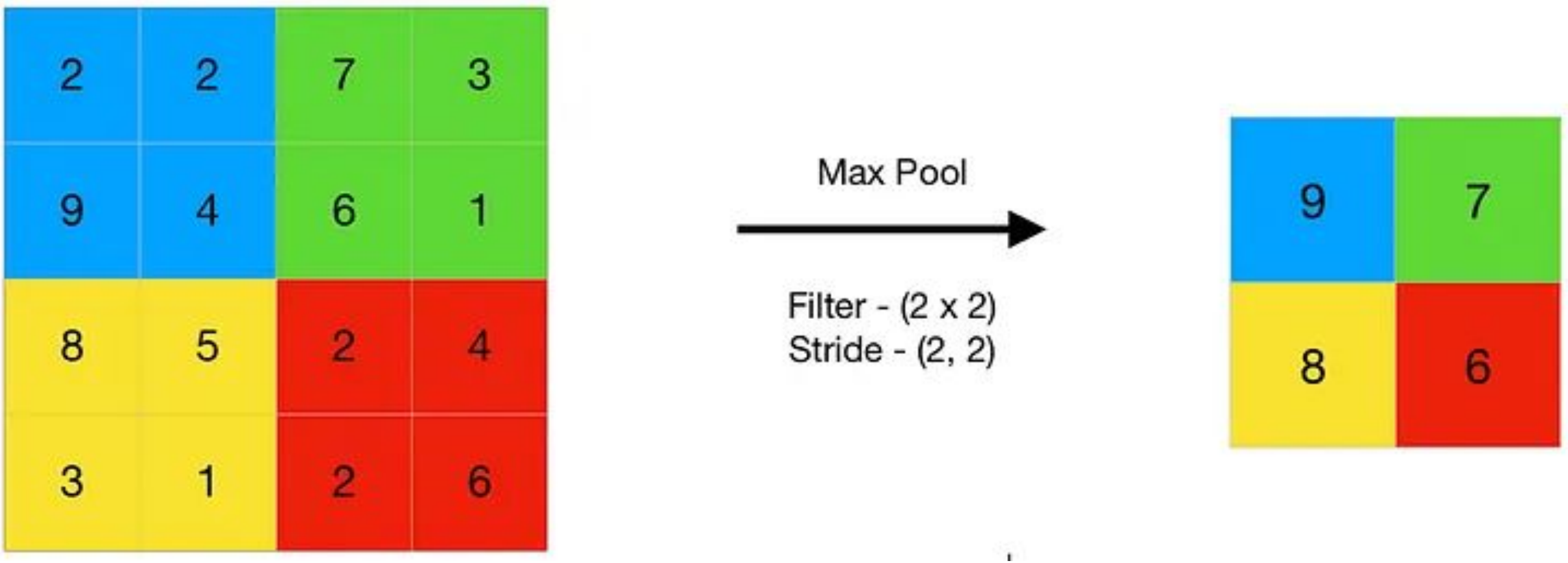
La capa de pulido reduce las dimensiones del mapa de características de la convolución.

Con esto se busca mantener las características de la imagen pero disminuyendo la dimensión espacial (clave en procesamiento).

Funciona de forma muy similar a una convolución, es decir, se aplica un filtro pero con el objetivo de reducir la dimensionalidad de la imagen. Los pesos no se alteran.

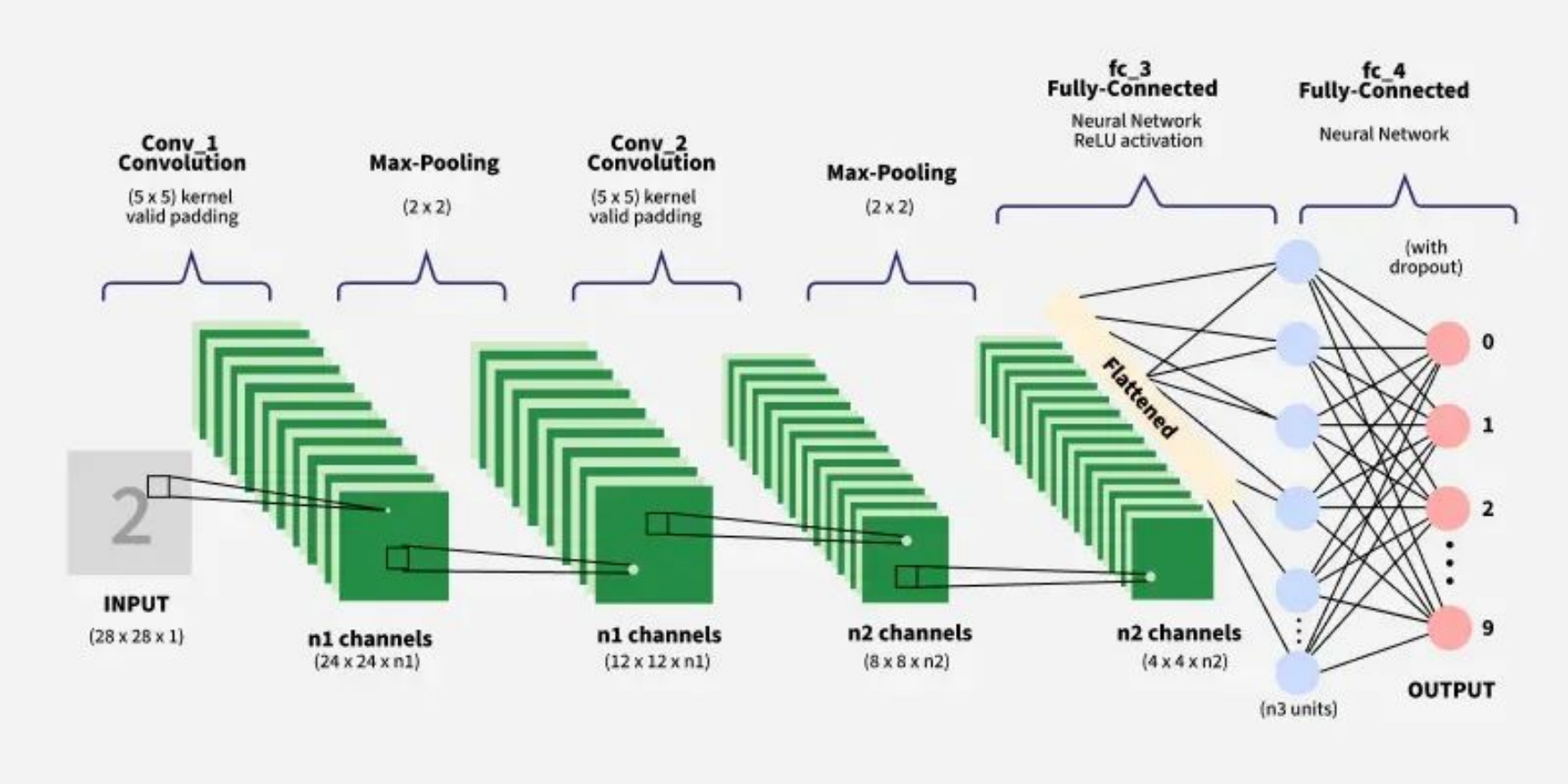


CNN - Redes Neuronales Convolucionales



CNN - Redes Neuronales Convolucionales

La capa totalmente conectada, permite realizar la clasificación de la imagen basada en la información del mapa de características que se obtuvo al aplicar las diferentes convoluciones.



CNN - Redes Neuronales Convolucionales

Existen diferentes arquitecturas de CNN populares. Lo bueno de estas arquitecturas es que están validadas y son fáciles de implementar. Algunas de ellas:

- AlexNet
- VGGNet
- GoogLeNet
- ResNet
- ZFNet

FIN

